

Санкт–Петербургский государственный университет
Факультет математики и компьютерных наук

МОЖАЕВ Василий Михайлович

Выпускная квалификационная работа

***Реализация библиотеки ввода/вывода с прозрачным
сжатием, эффективным произвольным доступом и
модификацией данных***

Уровень образования: бакалавриат

Направление 01.03.02 «Прикладная математика и информатика»

Основная образовательная программа СВ.5156.2022 «Современное
программирование»

Научный руководитель: доцент
кафедры информатики,
к. ф.-м. н., Н. Ю. Ловягин

Рецензент: доцент кафедры вычислительной
техники и информационных технологий
СПбГМТУ, к.т.н., доцент, О. Н. Петров

Санкт-Петербург
2026 г.

Оглавление

Введение.	4
Обзор литературы.	6
Постановка задачи.	11
Глава 1. Архитектура.	13
1.1. Общая архитектура.	13
1.2. Файл-контейнер.	14
1.3. Блочное сжатие и управление внутренней фрагментацией.	15
1.4. Индексация блоков: В-дерево.	16
1.5. Многофайловость.	18
1.6. Кэширование.	18
Глава 2. Реализация.	21
2.1. Технологический стек.	21
2.2. Документация.	22
2.3. API для языка C.	22
2.4. Кэширование.	26
2.5. В-дерево.	30
2.6. Тестирование.	32
Глава 3. Эксперименты.	34
3.1. Методика и тестовое окружение.	34
3.2. Борьба с внутренней фрагментацией.	35
3.3. Произвольное чтение: сравнение с аналогами.	37
3.4. Влияние кэширования и сравнение с Stdio.	43
3.5. Влияние размера блока и алгоритма сжатия.	45
3.6. Анализ накладных расходов.	47
3.7. Итоги экспериментов.	49
Выводы.	54
Заключение.	57
Список литературы.	58

Приложение А. Ссылки на исходный код и документацию	61
---	----

Введение.

С ростом объёмов обрабатываемых данных в биоинформатике, астрономии, машинном обучении и других областях задачи эффективного хранения и доступа к информации становятся всё более актуальными. Распространённые форматы сжатия, ориентированные на последовательную обработку (такие как `gzip`, основанный на алгоритме `Deflate`[10], `LZ4`[7], и другие), требуют полной декомпрессии файла для доступа к произвольному фрагменту данных, что неприемлемо для многих приложений. В биоинформатике существуют специализированные решения, обеспечивающие произвольный доступ к сжатым данным (например, `CRAM`[11]), однако они привязаны к конкретному типу данных. Более универсальные инструменты (`Seekable Zstd`[12], `Zran`[2]) поддерживают произвольное чтение, но не предоставляют эффективных механизмов записи, вставки или удаления данных внутри сжатого потока.

Между тем в ряде задач требуется не только перезапись отдельных фрагментов, но и вставка или удаление данных в середине файла с изменением его размера. В качестве примеров можно привести следующие сценарии. При работе с большими каталогами товаров в интернет-магазине (десятки миллионов записей) может потребоваться добавить новую запись в середину отсортированного списка или удалить устаревшую — при этом пережатие всего многогигабайтного файла было бы крайне неэффективно. В системах журналирования (логов) нередко возникает необходимость удалить из середины архивного лога записи за определённый период, не пересжимая остальные данные. В биоинформатике при работе с выравниваниями генома может потребоваться вставить или удалить отдельные выравнивания в сжатом файле без его полной переупаковки. Во всех этих случаях требуется не просто перезапись существующих данных, а изменение размера сжатого потока в произвольной позиции — операция, которую существующие решения общего назначения не поддерживают.

Помимо этого, выбор алгоритма сжатия существенно влияет на степень сжатия и скорость работы. Существуют как универсальные алгоритмы (`Zlib`[22], `LZ4`[7], `Zstd`[6], `Brotli`[3]), так и решения, заточенные под конкретные типы данных (например, `CRAM`[11] для геномных выравниваний, тайло-

вое сжатие FITS[20] для астрономических изображений). Желательно иметь инструмент, который не привязан к одному алгоритму, а даёт возможность подключать любой, в том числе собственный.

Целью данной работы является разработка библиотеки `Compio` (исходный код можно найти в публичном репозитории на GitHub, см 3.7), обеспечивающей прозрачное сжатие с произвольным доступом на чтение, запись, вставку и удаление, с интерфейсом, аналогичным `Stdio`, поддержкой нескольких файлов в одном архиве и возможностью подключения произвольных алгоритмов сжатия.

Разработка выполнялась в команде, и настоящая работа посвящена тем компонентам, которые выполнены автором: блочное сжатие данных с управлением внутренней фрагментации, индексация на основе B-дерева с массовыми операциями, система кэширования, поддержка подключаемых алгоритмов сжатия, API для языка C и экспериментальное исследование производительности библиотеки, в том числе и сравнение с альтернативными решениями. Остальные же компоненты были выполнены Д. В. Корбелайнен: низкоуровневый аллокатор блоков внутри файла-контейнера, объектно-ориентированное API для языка C++, обеспечение потокобезопасности и механизм упреждающей записи (WAL) для защиты от сбоев.

Обзор литературы.

В данной главе рассматриваются существующие алгоритмы и системы сжатия, поддерживающие произвольный доступ к данным, а также анализируются их ограничения.

Классические алгоритмы сжатия.

Большинство популярных алгоритмов сжатия (например, Deflate[10], Brotli[3], LZ4[7], Zstd[6]) работают как непрерывный поток кодов. В такой организации произвольный доступ к сжатым данным невозможен без полной распаковки всего потока с начала, что делает работу с большими файлами неэффективной.

Одним из распространённых решений является разбиение входных данных на блоки, каждый из которых сжимается независимо. Это позволяет распаковывать только нужный блок, не затрагивая остальные. Многие форматы архивов (например, xz[21], RAR[19] и 7z[1]) поддерживают несплошной (non-solid) режим сжатия, в котором каждый файл или заданная группа данных сжимается в отдельный независимый блок. Формат архива также содержит глобальный индекс — таблицу, в которой для каждого блока хранится его смещение в сжатом файле и исходный размер. При наличии такого индекса для доступа к произвольному фрагменту достаточно по таблице найти нужный блок и распаковать только его.

Однако даже при блочной организации произвольная *запись* (вставка, удаление или замена данных внутри блока) остаётся проблематичной. Изменение содержимого блока почти всегда меняет его сжатый размер, что приводит к сдвигу всех последующих блоков и требует перестройки их индексов и смещений. На практике любая модификация сжатых данных без полной перепакетки хотя бы затронутого блока и всех последующих оказывается невозможной. Поэтому для поддержки эффективной записи и редактирования требуются более сложные структуры данных и форматы хранения, выходящие за рамки классических алгоритмов.

Универсальные системы с произвольным чтением.

Существуют надстройки, которые используют существующие алгоритмы сжатия и предоставляют произвольный доступ к данным без полной декомпрессии. Они реализуют дополнительные методы (индексацию, сохранение состояния декомпрессора и т.п.) поверх стандартных сжатых потоков.

Zran.

Zran[2] — пример из дистрибутива Zlib, добавляющий произвольное чтение к потоку Deflate. Он строит индекс из точек останова (*break points*), для каждой из которых сохраняется полное состояние декомпрессора (скользящее окно, позиции в битовом потоке). При запросе на чтение распаковка начинается с ближайшей предыдущей точки останова, что требует декомпрессии лишь некоторого фрагмента. Индекс не требует модификации исходных сжатых данных и создаётся отдельно.

При формировании файла, поддерживающего произвольное чтение, сам индекс увеличивает общий размер архива. Накладные расходы зависят от частоты точек останова и объёма сохраняемого состояния: чем чаще точки, тем быстрее доступ, но тем больше индекс. Интервал между точками останова должен значительно превышать размер окна (по умолчанию в Deflate — 32 КиБ), иначе индекс становится чрезмерно большим. С другой стороны, слишком редкие точки останова приводят к необходимости распаковывать большие объёмы данных при каждом обращении, что замедляет доступ.

Такой подход оправдан в сценариях, где запросы на чтение имеют объём, сопоставимый с интервалом между точками останова и существенно превышают размер окна. Таким образом, Zran ориентирован на крупные пакетные чтения, а интервал точек останова следует выбирать соответствующим образом.

Seekable Zstd.

Seekable Zstd[12] — надстройка над алгоритмом сжатия Zstd, добавляющая произвольное чтение. Исходные данные разбиваются на независимые

кадры (frames), каждый из которых сжимается обычным Zstd с полным сбросом состояния. Для обеспечения произвольного доступа при формировании архива создаётся индекс, который хранит для каждого кадра его смещение в сжатом потоке и размер исходных данных. Индекс представляет собой простой статический массив, поиск по которому осуществляется с помощью бинарного поиска. При запросе на чтение по произвольной позиции библиотека по индексной таблице определяет кадры, содержащий нужные данные, и распаковывает только эти кадры.

Размер индексной таблицы пропорционален количеству кадров. Выбор размера кадра определяет компромисс: маленькие кадры дают более быстрый доступ к случайным позициям (распаковывается меньше данных), но снижают степень сжатия из-за потери межкадровой избыточности и увеличивают накладные расходы на индекс. Крупные кадры повышают степень сжатия, но требуют распаковки большего объёма данных при каждом обращении. Однако, в отличие от Zran, размер блока не обязан быть достаточно большим, так как размер индекса не растёт так сильно, что делает его пригодным для работы даже при маленьких операциях чтения.

Важно заметить, что хотя Seekable Zstd реализован конкретно для Zstd, подобный подход с разделением на блоки является вполне универсальным и может быть реализован с любым алгоритмом сжатия (примерами служат упомянутые выше форматы xz, RAR и 7z, поддерживающие несплошной режим). Однако ни один из выше упомянутых инструментов (Seekable Zstd и Zran) не поддерживает запись (только чтение), потому что для этого пришлось бы переупорядочивать сжатые блоки данных внутри архива, так как после записи они меняются в размере, но тогда возникла бы проблема внешней фрагментации (пустые места, оставшиеся от изменённых блоков), для решения которой требуются дополнительные подходы.

Узкоспециализированные решения.

В некоторых прикладных областях, где произвольное чтение больших объёмов сжатых данных критически важно, были созданы специализированные форматы. Они по-прежнему используют разбиение на независимо сжима-

емые блоки и строят индекс для навигации, однако структура блоков жёстко привязана к семантике данных (тайлы изображений, группы строк, сортировка по координатам и т.п.). Такая организация позволяет достичь высокого компромисса между степенью сжатия и скоростью доступа для типичных запросов, но одновременно делает форматы принципиально не предназначенными для произвольной записи: любое изменение содержимого меняет сжатый размер блока и нарушает инварианты размещения, требуя дорогостоящего перестроения значительной части файла.

- **CRAM**[11] (биоинформатика) — формат для сжатых выравниваний ДНК относительно референсного генома. Данные сортируются по геномным координатам и группируются в контейнеры-блоки, что позволяет при запросе участка хромосомы распаковывать лишь небольшое число блоков. Однако вставка или удаление отдельного прочтения нарушает сортировку и изменяет сжатые размеры блоков, поэтому эффективная модификация без полной перепакровки невозможна.
- **LCQS**[13] (биоинформатика) — метод сжатия строк качества в FASTQ, учитывающий статистические свойства данных. Качество разбивается на блоки, для каждого подбирается модель сжатия, и строится индекс, позволяющий распаковывать произвольные диапазоны записей. Модификация строки качества внутри блока меняет его сжатое представление и разрушает структуру индекса; формат ориентирован исключительно на чтение.
- **FITS с тайловым сжатием**[20] (астрономия) — многомерные изображения разбиваются на прямоугольные тайлы в соответствии с пространственной структурой данных. Каждый тайл сжимается независимо (алгоритмы Gzip, Rice, PLIO и др.), что позволяет читать произвольную область неба, затрагивая только накрывающие её тайлы. Изменение фрагмента изображения перекодирует тайл в блок переменного размера, сдвигая все последующие тайлы и ломая внешний индекс; формат не предоставляет механизмов для частичной записи.

- **Apache Parquet**[4] — колоночный формат для аналитики больших данных. Данные разбиты на строковые группы (*row groups*) и хранятся по-колоночно; метаданные позволяют читать только нужные группы и столбцы, пропуская остальные. Формат проектировался как иммутабельный (*append-only*): любое изменение или вставка записи внутри группы требует перезаписи всей группы и обновления глобальных метаданных.

Таким образом, даже узкоспециализированные форматы, максимально использующие структуру данных для оптимизации произвольного доступа, не поддерживают эффективную вставку, удаление или замену данных внутри сжатого потока — как и рассмотренные универсальные системы с произвольным чтением, в которых любая модификация данных требует полной перепакетки всего архива из-за отсутствия механизмов управления фрагментацией и перераспределения блоков. Это подтверждает актуальность разработки универсальной библиотеки, которая сочетала бы прозрачное сжатие с полноценной произвольной записью и чтением.

Постановка задачи.

Целью работы является создание библиотеки `Compio`, удовлетворяющей следующим ключевым требованиям:

- интерфейс, аналогичный стандартной библиотеке ввода/вывода (`open/close/read/write/seek/tell`);
- эффективный произвольный доступ к сжатым данным, включая вставку и удаление данных из середины файла, с изменением его размера;
- возможность подключения произвольных алгоритмов сжатия;
- поддержка нескольких логических файлов в одном файле-контейнере.

Для достижения этой цели в рамках работы необходимо решить следующие **задачи**:

1. **Разработать формат файла-контейнера, поддерживающий многофайловость.** Формат должен обеспечивать хранение метаданных, индекса и сжатых блоков в одном самодостаточном файле, не требующем внешних ресурсов для переноса между системами, а также поддерживать *многофайловость* (в одном файле-контейнере может лежать несколько независимых *логических файлов*).
2. **Реализовать блочное сжатие с управлением внутренней фрагментацией.** Данные логического файла разбиваются на *логические блоки* переменного размера, сжимаемые независимо. Для предотвращения деградации производительности при длительных последовательностях вставок и удалений необходимо ввести механизмы слияния и разделения блоков для управления внутренней фрагментацией.
3. **Построить систему индексации для логических блоков.** Для отображения позиций логических файлов на физические адреса сжатых блоков требуется реализовать структуру данных типа ключ-значение. Она должна поддерживать не только точечные операции, но и эффективные массовые сдвиги ключей, что критично для операций вставки и удаления.

4. **Реализовать систему кэширования для минимизации количества операций сжатия и расжатия.** Для минимизации дисковых операций и вызовов компрессора необходимо разработать систему кэширования блоков, которая бы уменьшала количество операций сжатия и расжатия. Кэш должен сохранять согласованность при массовых сдвигах, возникающих при вставке/удалении, и гарантировать согласованность кэшированных данных с индексом при выполнении операций записи, вставки и удаления.
5. **Предоставить API на языке C с поддержкой подключаемых алгоритмов сжатия.** Пользовательский интерфейс должен быть представлен в виде набора функций на C, использующих непрозрачные дескрипторы. Должна быть предусмотрена возможность задания собственных функций сжатия/расжатия, а также сохранения типа алгоритма в заголовке контейнера для восстановления при открытии.
6. **Провести экспериментальное исследование.** Требуется измерить эффективность предложенных методов борьбы с внутренней фрагментацией и сравнить производительность Compio с аналогами (Seekable Zstd, Zran) в сценариях произвольного чтения, а также оценить влияние размера блока на компромисс между скоростью доступа и размером файла-контейнера.

Решение перечисленных задач составляет содержание последующих глав: в главе 1 рассматривается общая архитектура и принимаемые проектные решения, в главе 2 описываются детали реализации ключевых компонентов, а в главе 3 приводятся результаты экспериментов.

Глава 1. Архитектура.

1.1. Общая архитектура.

Архитектура продиктована вышеперечисленными требованиями к библиотеке: прозрачный интерфейс с поддержкой подключения своего алгоритма сжатия, эффективная индексация логических блоков, многофайловость.

- Интерфейс библиотеки буквально состоит из аналогов стандартных операций `open/close/read/write/seek/tell`, и с точки зрения пользователя нет практически никакой разницы между `Compio` и `Stdio` (кроме конфигурации и понятия многофайловости).
- Файл делится на логические блоки, каждый из которых сжимается независимо, и для их эффективной индексации используется *В-дерево* (обоснование данного выбора описано в разделе 1.4). Таким образом, для чтения/модификации какого-то участка данных необходимо распаковать только те блоки, которые непосредственно касаются этого участка.
- Алгоритм сжатия полностью вынесен из логики библиотеки в качестве абстракции, и от него требуются только 3 функции: `compress`, `decompress` и `get_buf_size`, которые пользователь может реализовать сам для любого алгоритма сжатия.
- Для того, чтобы хранить все нужные структуры в одном файле был реализован аллокатор блоков внутри файла¹, который позволяет хранить в одном файле множество независимых структур (заголовков, сжатые блоки разных файлов, узлы В-дерева).
- Для индексации внутри В-дерева используется составной ключ из идентификатора логического файла и позиции внутри него, что позволяет хранить все логические блоки в одном В-дереве, за счёт чего достигается многофайловость. Такое решение было выбрано в пользу двух

¹Низкоуровневый аллокатор блоков внутри файла-контейнера разработан Д. В. Корбелайнен. Детали его реализации выходят за рамки настоящей работы.

аргументов: снижается нагрузка на нижележащую файловую систему (в случае когда требуется работать с множеством логических файлов), а также упрощается перенос архива с одной машины на другую.

1.2. Файл-контейнер.

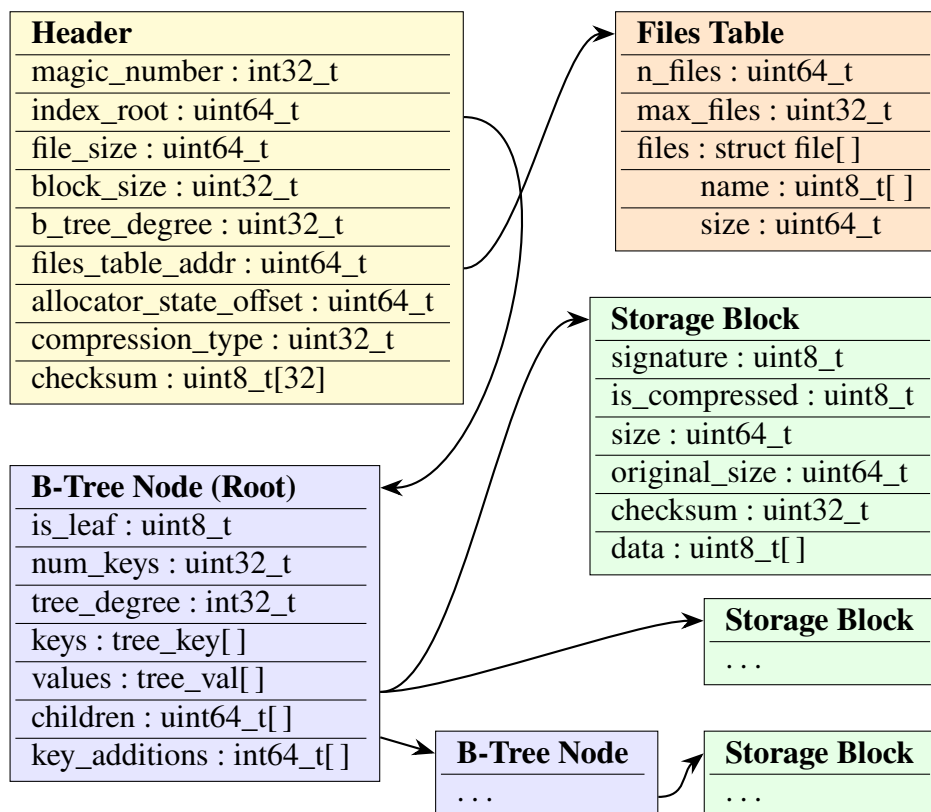


Рис. 1: Структуры внутри файла-контейнера и связи между ними.

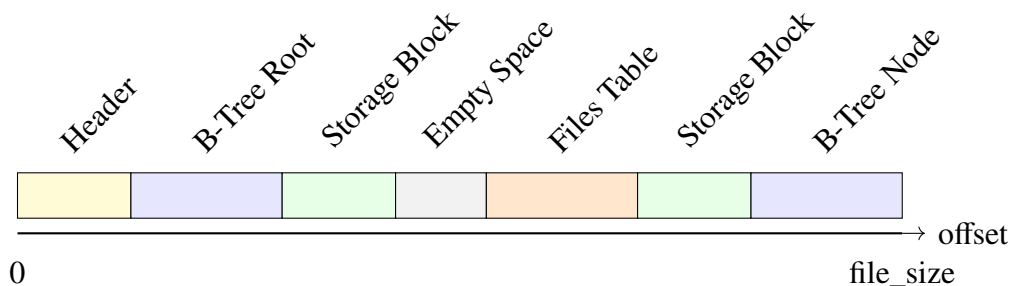


Рис. 2: Расположение структур внутри файла-контейнера. В начале расположен заголовок, а далее структуры расположены в произвольном порядке, который создается за счёт аллокатора.

Файл-контейнер (физический файл в файловой системе) представляет собой бинарный файл, в котором содержатся следующие структуры: заго-

ловок (конфигурация библиотеки, адрес узла В-дерева, список логических файлов), узлы В-дерева и сжатые блоки данных. Узлы В-дерева и сжатые блоки данных расположены вперемешку после заголовка, что возможно благодаря использованию аллокатора блоков в файле. Такой файл-контейнер полностью самодостаточен и легко подвергается переносу с одной машины на другую, так как содержит в себе всю необходимую для работы библиотеки информацию.

1.3. Блочное сжатие и управление внутренней фрагментацией.

Для поддержки эффективного произвольного доступа логический файл делится на блоки произвольного размера, каждый из которых сжимается независимо друг от друга. Так как библиотека должна поддерживать не только операции read/write, но и insert/erase (вставка и удаление), которые могут смещать существующие данные, простое деление файла на блоки фиксированного размера не сработает (при операциях вставки и удаления пришлось бы заново распаковывать все последующие блоки, заново производить разделение и снова сжимать). Поэтому было решено поддерживать блоки произвольного размера, что позволяет реализовать операции вставки и удаления эффективно.

Однако блоки произвольного размера приносят проблему внутренней фрагментации: блоки могут быть слишком маленькие (из-за чего их будет много, а значит потребуются более глубокое В-дерево и данные будут сжиматься хуже), либо слишком большие (из-за чего придется распаковывать большие блоки даже для самых маленьких операций, затрагивающих всего несколько байт). Это существенно замедляет работу библиотеки и делает ее более затратной по памяти.

Для решения проблемы внутренней фрагментации были введены три параметра конфигурации:

- `block_size` — целевой размер блока, который используется при создании новых блоков;
- `block_size_minimum` — минимальный размер блока, ниже которого

уходить нельзя (если появляется блок с таким размером, то производится попытка склеить его с соседним блоком);

- `block_size__maximum` — максимальный размер блока, выше которого уходить нельзя (если появляется блок с таким размером, он делится на несколько блоков).

Эти правила применяются во время операций модификации `write/insert/erase` по отношению к тем блокам, которые затрагиваются операцией. Например если `insert` вставляет данные внутрь существующего блока, то если суммарный размер не превосходит `block_size__maximum`, то новых блоков не создается, а только увеличивается существующий блок. Если же размер блока превзойдет `block_size__maximum`, то он делится на несколько блоков (для этого используется `block_size`) валидного размера.

1.4. Индексация блоков: В-дерево.

Обоснование выбора В-дерева.

Для каждой операции чтения или записи необходимо сначала понять какие блоки содержат нужные нам данные. В случае, если бы не было бы требования поддерживать операции вставки и удаления, то размеры блоков можно было бы сделать фиксированными, и тогда понять какие блоки нам нужны было бы нетрудно (чтобы получить номер блока по позиции в логическом файле нужно было бы просто поделить нацело его на размер блока). И тогда все, что нам бы оставалось сделать это сохранить в файле список физических адресов сжатых блоков внутри файла.

Однако в нашем случае блоки могут быть произвольного размера, и поэтому для индексации блоков внутри логических файлов используется **В-дерево**[5] — древовидная структура данных ключ-значение, оптимизированная под хранение своих узлов на диске (а не в оперативной памяти). Так как библиотека подразумевает работу с большими данными, структура данных для индексации также может быть большой, и потенциально она может не помещаться полностью в оперативную память. В-дерево отлично подходит для данной задачи, так как для работы с ним нам не нужно хранить его в опера-

тивной памяти, а можно только подгружать нужные узлы с диска, когда это требуется. Узлы В-дерева могут быть сколь угодно большими (это настраиваемый параметр), что позволяет сделать их размером со страницу блочного устройства, тем самым не замедлив операцию чтения одного узла, но сильно уменьшив глубину дерева, а значит и количество узлов, которые нужно прочитывать при запросах к В-дереву. Также В-дерево является сбалансированным по своему устройству, что гарантирует логарифмическую сложность операций к нему.

Поддержка многофайловости.

В В-дереве в качестве ключа используется пара (`file_id`, `pos`), где `file_id` — уникальный идентификатор логического файла, а `pos` — позиция внутри логического файла. В качестве значения же — (`addr`, `size`), где `addr` — адрес блока внутри файла-контейнера и `size` — размер блока. Таким образом для обслуживания всех логических файлов используется одно В-дерево и с точки зрения файла-контейнера нет никакой разницы между блоками из разных логических файлов, что упрощает архитектуру библиотеки.

Массовая операция смещения.

Для поддержки операций вставки и удаления необходимо иметь возможность смещать существующие данные. Использование древовидной структуры данных (В-дерево) для индексации блоков позволяет реализовать такую операцию достаточно эффективно (амортизированная логарифмическая сложность). Чтобы понять как, сначала рассмотрим наивную реализацию: чтобы сместить ключи в каком-то промежутке на какое-то смещение, мы рекурсивно проходимся по В-дереву, доходя до всех узлов, в которых содержатся ключи из промежутка, и обновляем эти ключи. Проблема в том, что в худшем случае нам нужно будет обратиться вообще ко всем узлам В-дерева.

Чтобы реализовать эту операцию эффективно был придуман и реализован подход с отложенными операциями. Идея заключается в том, что в каждом узле для каждого ребенка хранится смещение, которое означает, что все ключи соответствующего ребенка должны быть смещены на эту величину.

Вместо того, чтобы сразу смещать все элементы рекурсивно, мы распространяем смещение лениво (только тогда, когда мы спускаемся в соответствующего ребенка), из-за чего и достигается амортизированная логарифмическая сложность.

1.5. Многофайловость.

Как уже было замечено, в В-дереве используется составной ключ (`file_id`, `pos`). В результате в В-дереве ключи из разных логических файлов лежат последовательно и сгруппированно. Это позволяет поддерживать многофайловость без каких-либо дополнительных структур данных, кроме единого В-дерева.

Имена файлов и их размеры хранятся в таблице файлов — динамическом массиве, который располагается в контейнере и управляется аллокатором блоков аналогично узлам В-дерева и сжатым данным. Адрес таблицы хранится в заголовке контейнера. При добавлении нового файла таблица при необходимости расширяется, что позволяет поддерживать произвольное количество файлов в архиве. Явное хранение размера в таблице упрощает получение этой информации без обхода В-дерева.

1.6. Кэширование.

Наивная реализация каждой операции над логическим файлом приводит к многократным обращениям к диску и частым операциям сжатия/распаковки. Типичная операция чтения требует последовательного получения адреса корня В-дерева из заголовка, загрузки корневого узла, спуска по дереву до целевого листа (с чтением всех промежуточных узлов), затем чтения сжатого блока данных и его декомпрессии. Операция записи дополнительно включает сжатие модифицированных данных, управление местом в файле-контейнере через аллокатор, обновление адреса в узле В-дерева и, возможно, перебалансировку дерева.

С доступами к диску отчасти справляется страничный кэш операционной системы (`page cache`), однако операции сжатия и распаковки остаются доминирующими по времени выполнения. Именно они создают основную

вычислительную нагрузку при работе с архивом. Для снижения этих накладных расходов в библиотеке реализованы два независимых кэша: кэш узлов В-дерева и кэш распакованных блоков данных. Оба направлены в первую очередь на то, чтобы избежать повторного сжатия и распаковки одних и тех же данных.

Кэш узлов В-дерева.

Кэш узлов В-дерева хранит в оперативной памяти узлы, которые были недавно загружены с диска. Вытеснение производится по алгоритму *LRU* (Least Recently Used). Для повышения производительности в многопоточном окружении кэш является шардированным: он разбит на несколько независимых кэшей (шардов), каждый из которых защищён отдельным мьютексом. Эффективность кэша определяется тем, что корень В-дерева и узлы верхних уровней посещаются при каждом запросе к индексу. Если размер кэша превышает глубину дерева (а она логарифмически мала относительно объёма данных), эти узлы практически никогда не вытесняются, и большинство операций с индексом обходится без операций с диском.

Кэш блоков данных.

Кэш блоков данных хранит не сжатые данные, а уже распакованные фрагменты логических файлов. При обращении к некоторому участку логического файла соответствующий сжатый блок читается из контейнера, распаковывается и помещается в кэш. При высокой локальности доступа, когда множество последовательных операций приходится на одни и те же или соседние блоки, кэш блоков данных позволяет свести распаковку только к первому обращению к блоку. В предельном случае, когда рабочий набор данных полностью помещается в кэш, производительность чтения и записи приближается к работе с несжатыми данными в оперативной памяти: все операции обслуживаются из кэша, а сжатие и распаковка выполняются лишь единожды — при загрузке блока и при его вытеснении. Все последующие операции чтения или записи в пределах этого блока выполняются непосредственно в кэше без обращения к диску и, что важнее, без повторного сжатия или распаковки.

Именно за счёт сохранения распакованного состояния достигается главный выигрыш в производительности: затратные вычисления компрессора выполняются только один раз при загрузке блока и ещё раз при его вытеснении, если данные были модифицированы.

Вытеснение блоков из кэша производится по алгоритму *2Q-LRU*[17] (Two-Queue Least Recently Used). В отличие от классического LRU, *2Q-LRU* использует две очереди: *in-queue* (FIFO-очередь для новых элементов) и *main-queue* (LRU-очередь для часто используемых элементов). Новый блок при первом помещении в кэш попадает в *in-queue*. Если к нему происходит повторное обращение, он перемещается в *main-queue*, где дальнейшее вытеснение происходит уже по LRU. Такой подход защищает кэш от однократных «холостых» запросов: блок, который был прочитан всего один раз, не вытесняет часто используемые данные из основной очереди. При вытеснении блока из кэша изменённые данные сжимаются и записываются обратно в файл-контейнер. Таким образом, в случае попадания в кэш удаётся полностью избежать операций сжатия и распаковки, что критически важно для интерактивной работы с большими объёмами данных.

Глава 2. Реализация.

2.1. Технологический стек.

При разработке библиотеки одними из требований были кроссплатформенность и высокая производительность. Для обеспечения максимальной производительности желателен достаточно низкоуровневый язык. Чтобы предоставить возможность использования библиотеки из любых других языков программирования, необходим API на языке C: его ключевое преимущество — стабильный C ABI, который поддерживается практически любым языком через механизмы FFI (Foreign Function Interface).

Исходя из всех этих требований, внутренняя реализация библиотеки выполнена на языке C++ стандарта C++17. Использование C++17 позволило применить современные идиомы: умные указатели (`std::unique_ptr`, `std::shared_ptr`) для управления памятью, `std::optional` для безопасной обработки состояний, `std::map` для кэширования, а также `structured bindings` для улучшения читаемости кода. При этом пользователю предоставляется C API², что гарантирует совместимость и простоту интеграции.

Для автоматизации сборки выбран CMake (минимальной версии 3.15). CMake позволяет генерировать проектные файлы для различных сред (Makefiles, Ninja, Visual Studio) и управлять зависимостями. В конфигурации предусмотрены опции для включения/отключения отладочного вывода, а также возможность сборки библиотеки без операций вставки и удаления, что дает прирост производительности, если вышеупомянутые операции не требуются.

Для управления внешними зависимостями предоставлены файлы `vsrpkг.json` и `conanfile.txt` для систем пакетов `vsrpkг` и `Conan` соответственно. В качестве внешних зависимостей библиотека использует `Zlib`[22], `Zstd`[8], `LZ4`[7] и `Brotli`[14] — реализации соответствующих алгоритмов сжатия, которые пользователь может выбрать при конфигурации архива (см. 2.3). Для модульного тестирования и измерения производительности применяются `Google Test`[16] и `Google Benchmark`[15].

²Также Д. В. Корбелайнен был разработан API для C++, однако это выходит за рамки данной работы.

2.2. Документация.

API-документация генерируется с помощью Doxygen. Комментарии в заголовочном файле `compio.h` оформлены в формате, понятном Doxygen, что позволяет автоматически создавать HTML-версию справочного руководства. Также настроены GitHub Actions для автоматической генерации документации, которая лежит в публичном доступе (см. 3.7).

2.3. API для языка C.

Библиотека предоставляет низкоуровневый интерфейс на языке C, определённый в заголовочном файле `compio.h`. Все функции используют префикс `compio_` и оперируют двумя непрозрачными (opaque) типами-дескрипторами: `compio_archive` (открытый архив) и `compio_file` (открытый файл внутри архива). Такой подход обеспечивает строгую инкапсуляцию и позволяет использовать библиотеку в проектах на чистом C, а также в любых других языках, поддерживающих C-совместимые интерфейсы. Пример работы с библиотекой представлен в 1.

Открытие и закрытие архива.

Для работы с архивом вызывается функция

```
compio_archive *compio_open_archive(const char *fp, const char
    *mode, const compio_config *c);
```

Параметр `fp` задаёт путь к файлу архива, `mode` определяет режим доступа:

- "r" — открыть существующий архив только для чтения.
- "r+" — открыть существующий архив для чтения и записи.
- "w" — создать новый архив (если файл существует, он перезаписывается).
- "w+" — создать новый архив для чтения и записи.

Третий аргумент должен быть указателем на структуру конфигурации `compio_config` (см. 2.3). При успешном открытии возвращается указатель

на `compio_archive`, при ошибке — `NULL`, а код ошибки помещается в `errno`. Архив закрывается вызовом `compio_close_archive`, который сбрасывает все кэши, синхронизирует метаданные с диском и освобождает ресурсы.

Работа с файлами внутри архива.

Файл внутри архива открывается функцией

```
compio_file *compio_open_file(const char *name, compio_archive
    *archive);
```

Имя файла не должно превышать `COMPIO_FNAME_MAX_SIZE` (32 символа по умолчанию). Возвращаемый дескриптор `compio_file` используется для всех последующих операций чтения, записи и позиционирования. После завершения работы файл закрывается через `compio_close_file`.

Операции ввода-вывода.

Основные функции для работы с данными:

- `uint64_t compio_read(void *ptr, uint64_t size, compio_file *file)` — читает до `size` байт из текущей позиции файла в буфер `ptr`, возвращает количество реально прочитанных байт.
- `uint64_t compio_write(const void *ptr, uint64_t size, compio_file *file)` — записывает `size` байт из буфера `ptr` в текущую позицию файла, возвращает число записанных байт.
- `uint64_t compio_insert(const void *ptr, uint64_t size, compio_file *file)` — вставляет `size` байт из буфера `ptr` в текущую позицию файла, сдвигая последующие байты вправо.
- `uint64_t compio_erase(uint64_t size, compio_file *file)` — удаляет `size` байт с текущей позиции файла, сдвигая хвост файла влево.
- `int compio_seek(compio_file *file, int64_t offset, uint8_t origin)` — перемещает текущую позицию на `offset` байт относи-

тельно текущей позиции в файле, начала файла или конца файла (управляется аргументом `origin`).

- `uint64_t compio_tell(compio_file *file)` — возвращает текущую позицию в файле.

В случае ошибки все функции устанавливают в `errno` значение, соответствующее POSIX-стандарту: `ENAMETOOLONG`, `ENFILE`, `EROFS`, `EINVAL` и другие.

Конфигурация.

Структура `compio_config` объединяет все настраиваемые параметры библиотеки. В таблице 1 перечислены поля, относящиеся к данной работе.

Поле	Тип	Назначение
<code>compressor</code>	<code>compio_compressor</code>	Алгоритм сжатия (см. 2.3)
<code>b_tree_degree</code>	<code>int</code>	Степень В-дерева
<code>block_size</code>	<code>int</code>	Целевой размер блока (байт)
<code>block_size__minimum</code>	<code>int</code>	Минимальный размер блока (байт)
<code>block_size__maximum</code>	<code>int</code>	Максимальный размер блока (байт)
<code>cache_size__nodes</code>	<code>int</code>	Ёмкость кэша узлов В-дерева (количество)
<code>cache_size__blocks</code>	<code>int</code>	Ёмкость кэша блоков данных (количество)

Таблица 1: Поля структуры `compio_config`, относящиеся к данной работе. Остальные поля (`allocation_strategy`, `fill_holes_with_zeros`, `fragmentation_threshold`, `max_files`, `wal_max_size_bytes`, `checksum_type`, `wal_sync_mode`, `auto_batch_size`) относятся к компонентам, реализованным Д. В. Корбелайнен, и не рассматриваются в данной работе (однако полностью описаны в документации, см. 3.7).

Поля `b_tree_degree` и `block_size` пользователь может задать равными нулю, и тогда при открытии существующего архива они будут прочитаны из заголовка. Это поведение работает только в случае, когда открывается уже существующий архив, а не создаётся новый. Для удобства предоставлена функция `compio_build_default_config`, заполняющая структуру значениями по умолчанию.

Выбор алгоритма сжатия.

Для каждого поддерживаемого алгоритма реализована своя функция-конструктор вида `compio_build*_compressor`, где `*` — один из идентификаторов: `zlib`, `lz4`, `zstd`, `brotli`, а также `dummy` (отсутствие сжатия). Чтобы использовать собственный алгоритм, пользователь должен самостоятельно заполнить структуру `compio_compressor`, указав функции `compress`, `decompress`, `get_buf_size` и установив поле `compression_type` в `COMPIO_COMPRESS_CUSTOM`.

Созданный экземпляр `compio_compressor` помещается в поле `compressor` структуры `compio_config`. Затем эта конфигурация передаётся в функцию `compio_open_archive` при создании нового архива (режимы `"w"` и `"w+"`). Чтобы файл-контейнер был самодостаточным, и его можно было распаковать без дополнительной информации про алгоритм сжатия, библиотека сохраняет значение `compression_type` из компрессора в заголовке контейнера.

Перед открытием существующего архива пользователь может определить тип сжатия, вызвав функцию `compio_get_compression_type`. Затем он обязан самостоятельно инициализировать компрессор, соответствующий этому типу. Для встроенных алгоритмов это можно сделать с помощью вспомогательной функции `compio_build_compressor_by_type`. Для типа `COMPIO_COMPRESS_CUSTOM` пользователь должен предоставить собственную реализацию компрессора, идентичную той, что использовалась при создании архива.

Библиотека не создаёт компрессор автоматически и не изменяет поле `compressor` в переданной конфигурации. Если тип сжатия, прочитанный из заголовка архива, не соответствует типу, указанному в предоставленном пользователем компрессоре (например, архив требует `ZLIB`, а передан `LZ4`), или для кастомного типа передан некорректный компрессор, функция `compio_open_archive` вернёт ошибку. Такой механизм даёт пользователю полный контроль над процессом, но требует явных действий для согласования компрессора с архивом.

Внутри библиотеки все операции сжатия и разжатия выполняются ис-

ключительно через вызовы функций, на которые указывают поля `compress`, `decompress` и `get_buf_size`. Это позволяет абстрагироваться от конкретного алгоритма и использовать любые кодеки, в том числе и предоставленные пользователем.

```
#include "compio.h"

int main() {
    compio_config cfg;
    compio_build_default_config(&cfg);
    compio_build_lz4_compressor(&cfg.compressor);

    compio_archive *arch = compio_open_archive("data.bin",
                                              "w+",
                                              &cfg);
    compio_file *f = compio_open_file("A", arch);

    const char *data = "Hello, world!";
    compio_write(data, 13, f);

    char buf[13];
    compio_seek(f, 0, COMPIO_SEEK_SET);
    compio_read(buf, 13, f);

    compio_close_file(f);
    compio_close_archive(arch);

    return 0;
}
```

Листинг 1: Пример использования библиотеки `Compio` через API на C.

2.4. Кэширование.

Как было показано выше, каждая операция над логическим файлом потенциально требует множества обращений к диску: чтение заголовка, загрузка узлов В-дерева по пути от корня до листа, чтение сжатых блоков данных, а при записи — дополнительное сжатие, запись новых блоков и обновление узлов. Без кэширования производительность падает катастрофически даже при умеренных размерах архива. Для решения этой проблемы в библиотеке реализованы два независимых кэша: кэш узлов В-дерева и кэш распакованных блоков данных.

Кэш узлов В-дерева.

Кэш для узлов использует шардированный кэш с алгоритмом вытеснения LRU (Least Recently Used). Шардированность означает, что кэш разбит на несколько независимых кэшей (шардов), каждый из которых защищен мьютексом. При обращении в кэш сначала определяется нужный шард через хэш-функцию, а потом уже захватывается нужный мьютекс и идет работа с самим кэшем. Использование шардирования позволяет улучшить производительность в случае многопоточности.

Кэш реализован шаблонным классом `sharded_lru_cache`. Каждый из шардов представляет собой классический LRU-кэш на основе `std::unordered_map` и `std::list` (используется шаблонный класс `unordered_lru_cache`). Использование `std::unordered_map` вместо `std::map` позволяет выполнять операции за $O(1)$ (вместо $O(\log n)$).

Ключом служит 64-битный адрес узла внутри файла-контейнера, значением — умный указатель `smart_infile_object<index_node>`. `index_node` наследуется от абстрактного класса `infile_object`, который определяет функции, необходимые для сериализации/десериализации объекта. `smart_infile_object` — шаблонный класс, схожий по своей структуре с `shared_ptr` из стандартной библиотеки, но работающий с объектами типа `infile_object`.

Такая архитектура на основе шаблонных классов позволяет значительно упростить работу со структурами, которые могут одновременно лежать и в файле и в оперативной памяти, что может требовать синхронизации. Идея заключается в том, что если структура типа `infile_object` (а значит в том числе и `index_node`) модифицируется, то у нее выставляется флаг `is_modified`, который сигнализирует о том, что экземпляр объекта в оперативной памяти отличается от его копии в файле. В таком случае, до тех пор пока этот объект лежит в кэше и никто не пытается прочитать его некорректную (старую) версию из файла, а берет его из кэша, проблем не возникает. Когда же происходит вытеснение из кэша, в случае когда выставлен флаг `is_modified`, `smart_infile_object` вызывает функцию для десериализации объекта обратно в файл.

Таким образом, количество операций доступа к диску минимизируется. Кэш становится наиболее эффективным, когда пользователь часто делает запросы в близкие места в логическом файле, потому что тогда в кэше сохраняются все нужные узлы и запросов к диску вообще не происходит.

Кэш блоков данных.

Кэш для блоков данных позволяет не только уменьшить количество операций доступа к диску, но и операций сжатия/расжатия. Это становится возможным благодаря тому, что блоки данных хранятся в кэше в распакованном виде.

Значением в кэше является `smart_infile_pointer<block>`, где `block` наследуется от `infile_object`. В своих методах сериализации/десериализации он не только записывает/читает из файла, но и сжимает/расжимает данные. Таким образом, когда блок хранится в кэше, он хранится в расжатом виде. Если же блок изменился и его требуется сохранить в файл, то данные сжимаются и далее происходит аллокация нового региона внутри файла-контейнера, а старый адрес деаллоцируется³. Это необходимо, потому что если блок изменился, то и его размер также поменялся (потому что хранятся данные в сжатом виде).

Ключом же в кэше является структура `tree_key`, которая является парой из двух полей типа `uint64_t`: `file_id` — уникальный идентификатор логического файла, получаемый из инкрементального счетчика в заголовке при создании логического файла, `pos` — начальная позиция блока внутри логического файла. Как будет замечено позже, в B-дереве в качестве ключа используется та же самая структура `tree_key`.

В качестве алгоритма вытеснения используется *2Q-LRU* (Two-Queue Least Recently Used, см. 1.6), реализованный шаблонным классом `lru_2q_cache` на основе `std::map` и двух `std::list`.

Причина использования `std::map` вместо `std::unordered_map` заключается в том, что для кэша блоков критически необходима операция смещения целого региона блоков на какое-то количество байт в логическом файле.

³Интерфейс аллокатора (выделение и освобождение блоков) предоставлен Д. В. Корбелайнен; в данной работе используется как готовый компонент.

Эта операция нужна при выполнении операций вставки и удаления, так как нам нужно не только сместить данные в В-дереве, но и сместить их в кэше, чтобы записи в нем оставались корректными. Для такой массовой операции смещения на отрезке необходима `std::map`, потому что в ней можно использовать `std::map::lower_bound` и `std::map::upper_bound`, чтобы операция смещения не занимала линейное время от количества записей в кэше (а линейное только лишь от количества блоков, которое нужно сместить).

Временный индекс.

Как уже было отмечено, аллокация региона в файле для блока данных происходит только при вытеснении его из кэша. А значит что до тех пор, пока блок лежит в кэше, мы не знаем его адрес в файле-контейнере (что логично, потому что мы не знаем, какой у него будет размер, когда он будет туда записан). Поэтому пока блок лежит в кэше, в В-дереве в качестве адреса записан либо старый адрес, либо 0 в случае новых блоков. Само по себе это не кажется проблемой, потому что адрес блока внутри файла-контейнера нам нужен только в случае, если мы хотим его оттуда прочесть, однако этот адрес может быть невалидным только в случае, когда блок находится в кэше, а значит читать нам его из файла не нужно.

Однако эта логика ломается из-за того, как реализована операция записи (`compio_write`): так как она по своей сути модифицирует целый регион в файле, ей требуется изменить существующие блоки. Для этого она использует операцию В-дерева, которая находит все блоки в заданном диапазоне. Далее она проходится по получившемуся списку блоков и читает их из файла, используя адреса, которые выдало В-дерево. Однако запрос к В-дереву для получения блоков в диапазоне совершается единожды, до всех изменений данных в блоках. Поэтому возможна такая ситуация: допустим наш логический файл состоит из двух блоков А и В, из которых только блок В лежит в кэше (представим что размер кэша — 1). При операции записи, которая затрагивает оба блока, мы сначала делаем запрос к В-дереву, который выдает нам адреса обоих блоков. Сначала мы читаем блок А из файла, и, так как размер кэша — 1, блок В вытесняется из кэша, сжимается и записывается по

новому адресу, полученному от аллокатора. Однако после обработки блока А, мы хотим изменить блок В, но так как уже не лежит в кэше, нам нужно прочитать его из файла. Но адрес, который мы получили из В-дерева перед началом операции, некорректен, так как новый адрес блока В был аллоцирован только после этого. Таким образом, единственным вариантом получить новый адрес блока В является новый запрос к В-дереву.

Чтобы не делать лишних запросов к В-дереву в ситуациях, когда нам нужно обработать много блоков, был введен **временный индекс**. По своей сути это `std::map`, который дополняет настоящее В-дерево в сценарии, описанном выше (то есть сначала запрос идет в временный индекс, а при неудаче в В-дерево). Однако это не просто копия В-дерева, которая полностью хранится в оперативной памяти (это бы ломало всю суть библиотеки, которая заключается в работе с большими данными). Можно заметить, что временный индекс нам требуется только в исключительных случаях, когда мы обрабатываем несколько блоков, адреса которых получены единожды с помощью операции запроса на отрезке у В-дерева. И временной индекс нужен только для тех блоков, которые были вытеснены непосредственно во время обработки этого списка адресов, полученных единожды из В-дерева. А это значит, что после завершения обработки этого списка блоков мы можем очистить временный индекс, потому что после этого момента полученные некорректные адреса больше использоваться не будут, а будут запрашиваться непосредственно из В-дерева. И также, включать временный индекс необходимо только перед обработкой списка адресов, полученных из В-дерева, а в остальные моменты временный индекс можно отключить и не использовать. Поэтому размер временного индекса никогда не превышает количество блоков, которое обрабатывается в одной операции записи, а значит он является эффективной заменой В-дерева в этих исключительных случаях.

2.5. В-дерево.

Как обосновано в разделе 1.4, для индексации блоков произвольного размера используется В-дерево, ориентированное на хранение узлов на диске. В-дерево реализовано с нуля, а не взято в качестве внешней зависимости, по-

скольку библиотеке требуются специализированные массовые операции, отсутствующие в стандартных реализациях: получение всех пар ключ-значение в заданном диапазоне (`get_range`) и смещение ключей в диапазоне на заданную величину (`add_to_range`). Последняя реализована через механизм отложенных операций для обеспечения амортизированной логарифмической сложности. Ниже рассматриваются детали реализации: выбор степени, поддерживаемые операции и механизм отложенных операций.

Выбор степени В-дерева.

Степень В-дерева является настраиваемым параметром конфигурации библиотеки (`compio_config::b_tree_degree`). От него напрямую зависит ветвистость В-дерева, а значит и высота дерева, что в свою очередь влияет на скорость операций. Рекомендуется устанавливать такое значение степени дерева, чтобы размер одного узла был примерно равен странице устройства, на котором лежит файл-контейнер. Формула для размера узла В-дерева: $96 \cdot b_tree_degree - 16$. В случае 4 КиБ подходящее значение степени может быть 42, так как тогда размер узла составляет 4016 байт.

Важно заметить, что библиотека никак не выравнивает расположение узлов В-дерева внутри файла по границам страниц, а значит то, что размер узла меньше чем размер страницы гарантирует лишь то, что будет прочитано не более двух страниц (а не ровно одна). Поэтому достаточно лишь выбрать степень так, чтобы размер узла был примерно равен размеру страницы, а не строго меньше или равен ему.

Операции.

В В-дереве реализованы стандартные операции: вставка пары ключ-значение (`insert`), удаление по ключу (`remove`), получение значения по ключу (`get`), обновление значения по ключу (`update`). Для этих операций использовались стандартные алгоритмы В-дерева. Также реализованы массовые операции: получение всех пар ключ-значение в определенном диапазоне ключей (`get_range`) и смещение всех ключей в определенном диапазоне на некоторый сдвиг (`add_to_range`). `get_range` требуется для операций чтения или

записи, затрагивающих много блоков, а `add_to_range` для операций вставки и удаления из середины логического файла.

Отложенные операции.

Для эффективной реализации массовой операции смещения ключей (`add_to_range`) используется идея с отложенными операциями: в каждом узле хранятся смещения для каждого из его потомков, и при выполнении операции `add_to_range` находится такое подмножество узлов, которые ровно покрывают нужный диапазон, и смещения применяются только к элементам в них и их предках, а для детей лишь выставляются смещения. Когда какие-либо операции делают запрос к ребенку, для которого установлено ненулевое смещение, оно распространяется на этого ребенка (ключи в нем смещаются, и меняются смещения для его потомков, если он не лист). Таким образом, время работы такой операции является амортизированным логарифмическим, вместо линейного в наивной реализации.

2.6. Тестирование.

Функциональность библиотеки верифицируется набором модульных тестов, сфокусированных на ключевых компонентах: В-дерево, файловые операции и общие сценарии использования.

Тестирование В-дерева.

Проверяется корректность вставки, удаления, обновления и поиска ключей. Отдельное внимание уделено операциям с диапазонами: запрос на получение всех блоков, попадающих в интервал, а также массовый сдвиг ключей, необходимый для реализации вставки и удаления внутри логического файла. Тесты охватывают сценарии с разной степенью ветвления, переполнением узлов, слиянием и заимствованием элементов при удалении. Проверяется также работа с краевыми значениями (нулевые размеры регионов, максимальные позиции) и корректность кэширования узлов В-дерева.

Тестирование общих файловых операций.

Набор тестов проверяет базовые возможности: последовательную и произвольную запись/чтение, позиционирование, корректное восстановление данных после закрытия и повторного открытия архива. Для повышения надёжности выполняются параметризованные тесты с разными размерами блоков и объёмами данных. Специализированные тесты охватывают операции вставки и удаления фрагментов внутри файла: вставка в начало, середину, конец; удаление с начала, середины, конца; комбинации вставок и удалений; обработка нулевых размеров; корректное изменение позиции курсора после операций. Дополнительно используются случайные последовательности операций, где на основе эталонной модели в памяти проверяется идентичность содержимого файла после каждого действия. Такой подход позволяет выявить скрытые ошибки при чередовании записи, чтения, вставки, удаления и позиционирования.

Глава 3. Эксперименты.

3.1. Методика и тестовое окружение.

Эксперименты, требовавшие точных измерений времени, выполнялись либо с помощью фреймворка Google Benchmark[15], который подбирает число повторений автоматически для получения стабильных результатов, либо с помощью самописных бенчмарков, где требовался полный контроль над ходом эксперимента. Измерения проводились на стенде со следующими характеристиками:

- Центральный процессор: AMD Ryzen 5 5600 (6 ядер / 12 потоков), максимальная частота до 4,47 ГГц, кэш L3 32 МБ.
- Оперативная память: 32 ГБ DDR4-2666 (Kingston, 2×16 ГБ).
- Накопитель: SSD Kingston SNV3S1000G (NVMe, 1 ТБ).
- Материнская плата: Gigabyte B550M DS3H R2.
- Операционная система: Ubuntu 24.04.3 LTS, ядро 6.17.0-23-generic.

В качестве тестовых данных использовались 2 файла:

- файл `webster` из корпуса сжатия Силезского университета (Silesia compression corpus)[9], который содержит несжатый текст The 1913 Webster Unabridged Dictionary, сохранённый в HTML-разметке (размер составляет 39,5 МиБ, но в некоторых экспериментах он обрезался до 32 МиБ)
- файл `enwik9` из набора данных Large Text Compression Benchmark[18], содержащий первые 10^9 байт дампа английской Википедии в формате XML (размер 1 ГБ, но из него использовались различные непересекающиеся участки по 32 МиБ, чтобы получать вариативность по данным одного типа)

Если не оговорено иное, библиотека `Compro` использовалась со следующими параметрами конфигурации, принимаемыми по умолчанию:

- `block_size` = 8192 байт;
- `block_size_minimum` = `block_size` / 4 = 2048 байт;
- `block_size_maximum` = `block_size` × 4 = 32768 байт;
- `tree_degree` = 42 (размер узла B-дерева около 4 КиБ);

Там, где требовалось изолировать эффект от кэширования самой библиотеки, размер кэша блоков выставлялся равным 1. Системный страничный кэш не сбрасывался; поскольку все сравниваемые инструменты работали в одинаковых условиях, его влияние не нарушает честности сопоставления.

3.2. Борьба с внутренней фрагментацией.

Постановка эксперимента.

Для проверки эффективности стратегии управления размерами блоков с помощью границ `block_size_minimum` и `block_size_maximum` моделировался длительный поток случайных вставок и удалений внутри логического файла размером 32 МиБ. Сравнивались два варианта:

- **Наивный** — блоки не сливаются и не перераспределяются целенаправленно. При вставке в середину блока он разрезается на два; при вставке между блоками создаётся новый блок с размером, равным вставляемым данным; при удалении крайние затронутые блоки делятся, а внутренние удаляются; дополнение файла записывается блоками целевого размера `block_size`. Без механизма слияния размеры блоков неограниченно уменьшаются.
- **С диапазоном** — используются пороги `block_size_minimum` и `block_size_maximum` (значения по умолчанию, как указано выше): слишком маленькие блоки сливаются с соседними, слишком большие разделяются на несколько блоков (см 1.3).

Эксперимент состоял из 20 000 шагов. На каждом шаге выбирался целевой размер файла из нормального распределения $\mathcal{N}(32768, 4096^2)$ байт

(среднее 32 КиБ, стандартное отклонение 4 КиБ). Если текущий размер файла оказывался меньше целевого, выполнялась вставка фрагмента соответствующей длины, в противном случае – удаление. После каждого шага измерялись количество блоков и средняя скорость операций (МБ/с). Подобный эксперимент проводился 10 раз и по результатам бралось медианное значение (как для количества блоков, так и для скоростей), а после этого использовался медианный фильтр (размер окна – 1001) для уменьшения шума в данных.

Результаты.

На рис. 3 видно, что без регулировки число блоков монотонно растёт и после нескольких тысяч операций достигает десятков тысяч. Это приводит к углублению В-дерева, росту числа дисковых операций и падению скорости до ≈ 3 МБ/с. При использовании диапазона размеров количество блоков стабилизируется на уровне 300–400, а скорость сохраняется в пределах 20–30 МБ/с (небольшое снижение с течением времени может быть связано с внешней фрагментацией свободного пространства внутри файла-контейнера, однако основная деградация надёжно предотвращена).

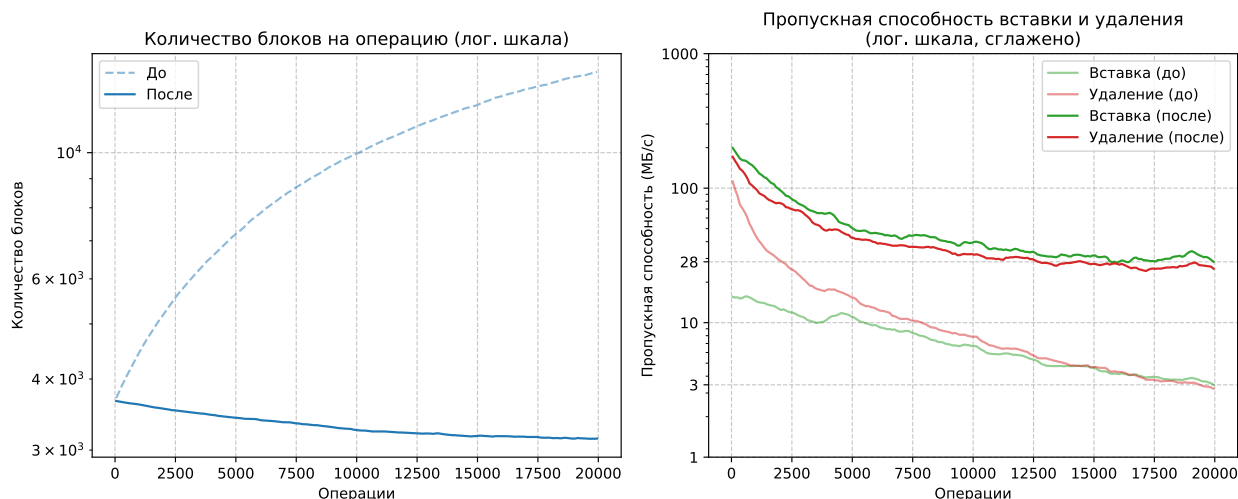


Рис. 3: Динамика количества логических блоков (слева) и средней скорости вставки/удаления (справа) в зависимости от числа выполненных операций. Наивная стратегия (без регулировки размеров блоков, тусклые линии на графике) приводит к неограниченному росту числа блоков и падению скорости до ≈ 3 МБ/с. Использование порогов `block_size_minimum` и `block_size_maximum` стабилизирует количество блоков на уровне 300–400 и удерживает скорость в диапазоне 20–30 МБ/с.

Вывод.

Механизм `block_size_minimum` и `block_size_maximum` надёжно предотвращает деградацию производительности, вызванную внутренней фрагментацией, и оправдывает проектное решение, заложенное в архитектуру библиотеки.

3.3. Произвольное чтение: сравнение с аналогами.

Краткое описание инструментов.

- *Seekable Zstd* разбивает входные данные на независимые кадры размером `max_frame_size`, каждый из которых сжимается алгоритмом Zstd с полным сбросом состояния. Индекс смещений кадров хранится в виде массива в оперативной памяти, поиск выполняется бинарным поиском. Принцип работы очень схож с принципом работы нашей библиотеки *Compio* (но в упрощенном виде, приспособленным только для произвольного чтения).
- *Zran* — надстройка над Deflate, использующая точки останова со снимками состояния декомпрессора (которые по размеру совпадают с размером окна, то есть 32 КиБ по умолчанию). Точки останова создаются с частотой `flush_interval`. При чтении распаковка начинается с ближайшей предыдущей точки, что делает метод более эффективным для крупных запросов (так как чтобы размер итогового архива был меньше размера исходных данных необходимо брать частоту точек останова значительно больше 32 КиБ).

Оба инструмента ориентированы только на произвольное чтение; любое изменение данных требует полной распаковки и повторной упаковки всего архива.

Для сравнения с этими двумя инструментами были проведены эксперименты с двумя конфигурациями библиотеки *Compio*: с алгоритмом сжатия Zstd (для *Seekable Zstd*) и с алгоритмом сжатия Zlib (для *Zran*). Были выбраны одинаковые уровни сжатия, чтобы проводить сравнение над конкретно

инструментами, а не нижележащими алгоритмами сжатия. Также в Compio был отключен кэш блоков, чтобы сравнение было честным (кэш узлов не был отключен, что соответствует поведению инструментов, у которых индексы лежат в оперативной памяти). Влияние кэширования на производительность отдельно рассматривается в 3.4.

Методика.

Архив размером 32 МиБ (HTML) подготавливался каждым из 4-х инструментов (использовались участки из enwik9, см. 3.1). Далее измерялся размер архива (МБ), который можно считать пригодным для произвольного чтения (то есть учитывается размер индекса). После этого проводилось 8192 операций чтения в произвольных местах файла, а размер каждого запроса выбирался из гамма-распределения с параметрами $\text{shape} = 4096$, $\text{scale} = 2$ (средний размер 8192 байт, стандартное отклонение 128 байт), и замерялась средняя пропускная способность (МБ/с).

Подобный эксперимент проводился с различными *параметрами гранулярности* у библиотек: `block_size` у Compio, `max_frame_size` у Seekable Zstd и `flush_interval` у Zran. В случае Compio+Zstd и Seekable Zstd проводились дополнительные эксперименты: выбирался наилучший параметр гранулярности по метрике $\frac{\text{проп. способность}}{\text{размер файла}}$ (им оказался 8192 для обеих библиотек), и с этим параметром проводились эксперименты с различными участками из enwik9. Для каждого участка данных вычислялось отношение обоих метрик (размер файла и пропускная способность) у двух библиотек, чтобы увидеть тренд вне зависимости от шума, возникающего при варьировании данных.

Результаты.

На рис. 4 сравниваются Compio (Zlib, уровень 1) и Zran. Каждая точка на кривых соответствует значению параметра гранулярности: для Compio — размер блока (512 байт – 2 МиБ), для Zran — интервал точек останова (32 КиБ – 1 МиБ). При малых значениях параметра архив получается большим, при больших — архив сжимается, но скорость падает из-за увеличения объёма данных, которые нужно распаковать для одного запроса (в среднем 8 КиБ).

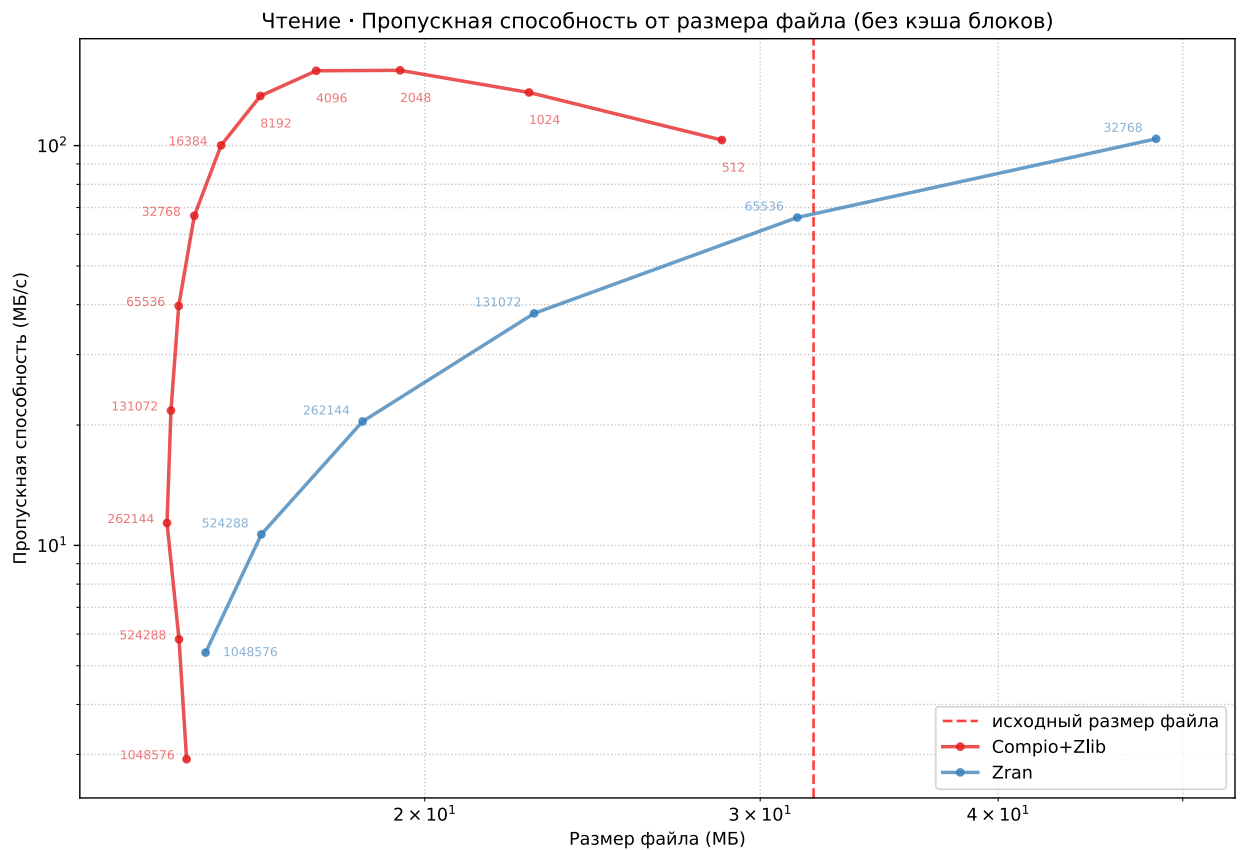


Рис. 4: Сравнение Compio (Zlib, уровень 1) и Zran: зависимость скорости произвольного чтения от размера архива (логарифмические шкалы). Каждая точка соответствует значению параметра гранулярности (размер блока для Compio, интервал точек останова для Zran). Compio достигает пиковой скорости 153 МБ/с при размере архива 17,5 МБ. Zran при интервалах от 256 КиБ даёт архив до 18,5 МБ и скорость не выше 20 МБ/с — Zran не оптимизирован для чтения блоков размером менее 256 КиБ и требует более крупных запросов для эффективной работы.

Compio+Zlib достигает пиковой скорости 153 МБ/с при размере архива 17,5 МБ, что соответствует умеренным значениям *block_size* (порядка 4–16 КиБ). Zran при любых практически приемлемых настройках (интервал от 256 КиБ) даёт заметно больший архив (18,5 МБ) и более низкую скорость (не выше 20 МБ/с). При росте интервала точек останова скорость Zran только снижается, что объясняется ростом объёма данных, которые необходимо распаковать для каждого запроса, и тем, что средний размер запроса гораздо меньше чем минимальный *flush_interval*. Для достижения сопоставимых результатов Zran требует иного сценария использования — например, крупных запросов (сравнимых с интервалом точек останова). В условиях же эксперимента со средним размером запроса 8 КиБ Compio+Zlib уверенно

превосходит Zran по всем ключевым метрикам.

Таким образом, основное различие между Comprio и Zran заключается в области применимости: Zran эффективен лишь при очень крупных запросах (мегабайты), тогда как Comprio за счёт настраиваемого размера блока адаптируется к любому паттерну доступа, обеспечивая высокую скорость даже при типичных запросах размером в несколько килобайт.

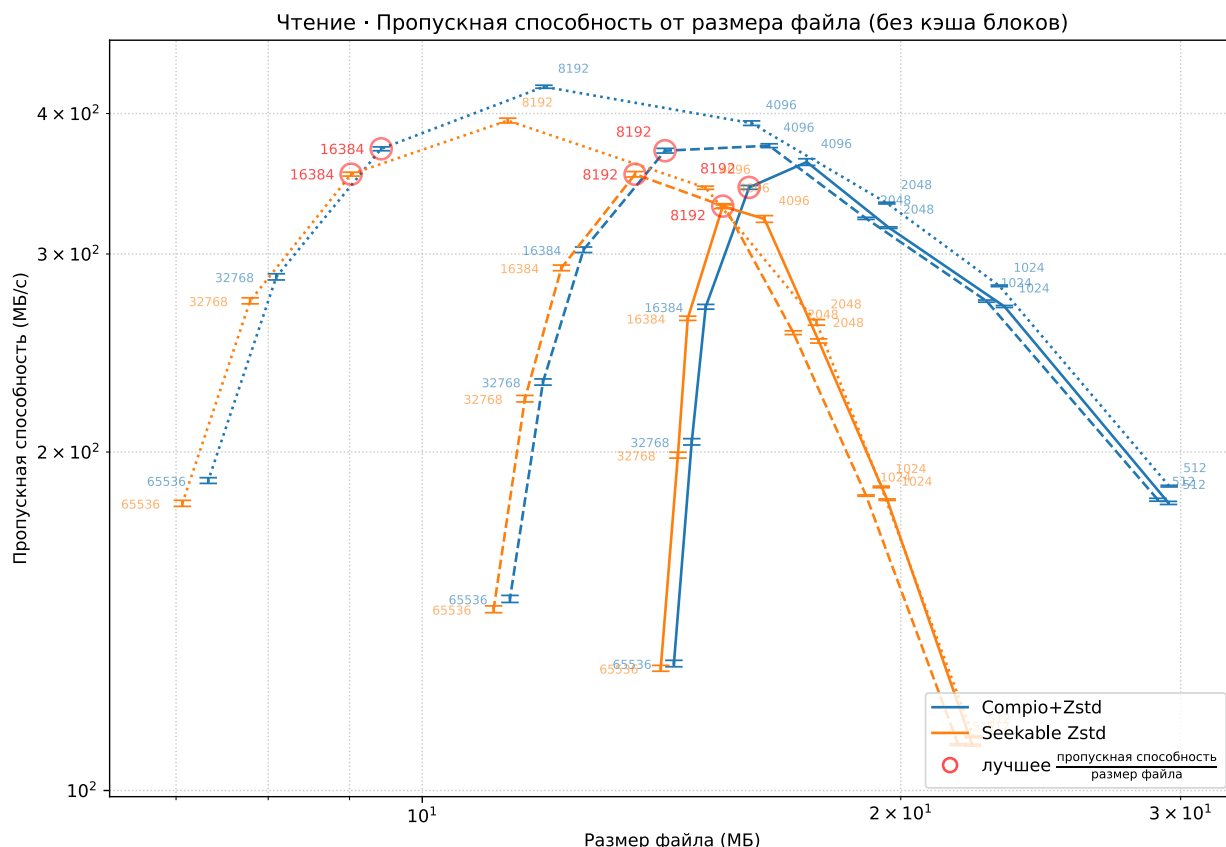


Рис. 5: Сравнение Comprio (Zstd, уровень 1) и Seekable Zstd на трёх участках enwik9[18]. Инструменты различаются цветом, участки — стилем линии. Точки — разные параметры гранулярности. Красные кружки — максимум отношения скорости к размеру архива. В области своих оптимальных для данного сценария использования параметров гранулярности (8–16 КиБ) инструменты практически идентичны.

На рис. 5 сравниваются Comprio+Zstd (уровень 1) и Seekable Zstd на трёх различных участках файла enwik9 (один из них соответствует центральной области файла, которая, как отмечено в описании тестовых данных LTСВ, характеризуется повышенной сжимаемостью[18]). Для каждого инструмента все линии имеют одинаковый цвет, а разные участки данных различаются стилем линии (сплошная, пунктирная, точечная). Точки на линиях — различным значениям параметра гранулярности (размер блока для Comprio, размер

Размер блока (байт)	Пропускная способность (МБ/с)					
	Регион 1		Регион 2		Регион 3	
	compio	s. zstd	compio	s. zstd	compio	s. zstd
512	180 ± 0,57	110 ± 0,22	181 ± 0,53	110 ± 0,20	186 ± 0,31	112 ± 0,19
1024	269 ± 0,56	181 ± 0,27	272 ± 0,60	183 ± 0,25	281 ± 0,45	186 ± 0,30
2048	317 ± 0,52	251 ± 1,07	323 ± 0,69	255 ± 0,99	333 ± 0,58	261 ± 1,49
4096	362 ± 2,45	322 ± 2,22	374 ± 1,47	331 ± 1,03	392 ± 1,82	344 ± 1,09
8192	344 ± 1,36	331 ± 1,65	371 ± 1,77	353 ± 1,95	423 ± 1,31	394 ± 1,94
16384	269 ± 1,31	263 ± 1,14	303 ± 1,71	292 ± 1,68	372 ± 1,47	353 ± 1,38
32768	204 ± 1,33	199 ± 1,20	231 ± 1,46	223 ± 1,48	286 ± 1,81	273 ± 1,68
65536	130 ± 0,86	128 ± 0,77	148 ± 1,07	145 ± 1,01	189 ± 1,10	180 ± 1,11

Таблица 2: Точные значения пропускной способности (МБ/с) для Compio (Zstd, уровень 1) и Seekable Zstd на трёх участках enwik9 при различных размерах блока. Рядом со средними значениями указано стандартное отклонение, соответствующее «усам» на рис. 5. Compio немного опережает Seekable Zstd по скорости на всех конфигурациях; при оптимальных размерах блока (4–16 КБ) преимущество составляет 4–12% в зависимости от участка.

Размер блока (байт)	Размер файла (МБ)					
	Регион 1		Регион 2		Регион 3	
	compio	s. zstd	compio	s. zstd	compio	s. zstd
512	29,5	22,2	29,0	21,7	29,5	22,2
1024	23,2	19,6	22,6	19,0	23,1	19,4
2048	19,7	17,8	19,0	17,1	19,6	17,7
4096	17,5	16,4	16,5	15,5	16,1	15,1
8192	16,1	15,5	14,2	13,6	11,9	11,3
16384	15,1	14,7	12,6	12,2	9,4	9,0
32768	14,8	14,5	11,9	11,6	8,1	7,8
65536	14,4	14,1	11,4	11,1	7,3	7,1

Таблица 3: Точные значения размера архива (МБ) для Compio (Zstd, уровень 1) и Seekable Zstd на трёх участках enwik9 при различных размерах блока. Seekable Zstd обеспечивает немного меньший размер архива на всех конфигурациях; при оптимальных размерах блока (4–16 КБ) архив Compio больше на 4–7% в зависимости от участка.

кадра для Seekable Zstd). Красными кружками отмечены точки, в которых отношение $\frac{\text{проп. способность}}{\text{размер файла}}$ максимально для каждого инструмента и каждого участка. Точные численные значения пропускной способности со стандартными отклонениями и размера архива для всех конфигураций приведены в табл. 2 и 3 соответственно.

Для обоих инструментов наилучший баланс достигается при размере блока/кадра 8192 или 16384 байт. В этой области кривые Compio+Zstd и Seekable Zstd практически совпадают как по скорости чтения, так и по размеру архива. При малых блоках (512–1024 байт) оба инструмента показывают низкую скорость из-за высоких накладных расходов; при больших (128 КиБ и выше) скорость падает, а выигрыш в размере архива становится незначительным. Таким образом, по ключевой метрике баланса между скоростью и степенью сжатия Compio+Zstd не уступает специализированному инструменту Seekable Zstd.

На рис. 6 представлено сравнение Compio+Zstd и Seekable Zstd на 29 регионах enwik9. Для каждого региона зафиксирован параметр гранулярности 8192 байт (оптимальный для обоих инструментов). По оси абсцисс отложено отношение размеров архивов (Compio / Seekable Zstd), по оси ординат — отношение скоростей чтения (Compio / Seekable Zstd). Красная диагональ $y = x$ соответствует равенству показателей. Зелёная область (выше диагонали) соответствует случаю, когда отношение $\frac{\text{проп. способность}}{\text{размер файла}}$ у Compio больше, чем у Seekable Zstd; розовая (ниже диагонали) — когда больше у Seekable Zstd. Это не означает, что Compio лучше по каждому параметру в отдельности, а лишь показывает, какой инструмент даёт больше пропускной способности на единицу размера архива.

Большинство точек лежат в розовой области, но все они расположены вблизи диагонали, и, что более важно, вблизи точки (1, 1), что означает что разница между инструментами минимальная. Типичные значения: отношение скоростей составляет 1,03–1,08 (то есть Compio быстрее на 3–8%), а отношение размеров — 1,04–1,05 (архив Compio на 4–5% больше). Соответственно, отношение $\frac{\text{проп. способность}}{\text{размер файла}}$ у Seekable Zstd оказывается немного выше (розовая область), но разница минимальна.

Оценка погрешностей.

Для каждого из 29 регионов enwik9 было выполнено 8192 операции произвольного чтения. По этим замерам вычислялись средняя пропускная способность v и её стандартное отклонение σ_v для Compio и Seekable Zstd (приведены в табл. 2). Размер архива для каждого региона и инструмента является детерминированной величиной (зависит только от данных и размера блока), поэтому погрешность для него не определяется. Для отношения скоростей $r = v_{\text{compio}}/v_{\text{s.zstd}}$ стандартная погрешность вычислялась по формуле распространения ошибок для частного независимых величин:

$$\sigma_r = r \sqrt{(\sigma_{\text{compio}}/v_{\text{compio}})^2 + (\sigma_{\text{s.zstd}}/v_{\text{s.zstd}})^2}$$

Вывод.

Даже при отключённом кэшировании Compio значительно превосходит Zran в типичном сценарии произвольного чтения с запросами размером в несколько килобайт (до 153 МБ/с против <20 МБ/с), поскольку Zran эффективен лишь при очень крупных запросах (мегабайты), тогда как Compio за счёт настраиваемого размера блока адаптируется к любому паттерну доступа. В сравнении с Seekable Zstd при оптимальном размере блока разница минимальна (около 4% в обе стороны): Compio немного быстрее, но архив Seekable Zstd несколько меньше. При этом Compio остаётся единственным решением, которое дополнительно поддерживает полноценную запись, вставку и удаление данных без перепакетки всего архива.

3.4. Влияние кэширования и сравнение с Stdio.

Постановка.

Данный эксперимент демонстрирует роль кэша распакованных блоков при высокой локальности обращений и позволяет сравнить скорость работы со стандартным несжатым вводом-выводом (Stdio с буферизацией).

Модель локальности строилась следующим образом: выбирался рабочий регион размером $2^{17} = 131072$ байт (128 КиБ), внутри которого случай-

ным образом позиционировался запрос на чтение/запись, причём размер запроса извлекался из гамма-распределения с $\text{shape} = 4096$, $\text{scale} = 2$ (среднее 8192 байт). Каждые `n_switch` операций регион менялся на другой, выбранный случайно во всём файле. Параметр `n_switch` варьировался по степеням двойки от 1 до 512, что позволяло получить широкий диапазон процентов попаданий в кэш (от практически нулевого до более 99%). `Compio` использовал стандартные настройки кэша блоков: ёмкость была достаточна для хранения всех блоков рабочего набора, поэтому вытеснения происходили только при смене региона; таким образом, процент попаданий определялся исключительно паттерном доступа. Для каждого уровня локальности измерялась средняя скорость (МБ/с). `Compio` тестировался с компрессорами LZ4, Zstd, Zlib и Brotli.

Для фиксированного процента попаданий 97,5% строился дополнительный график, показывающий компромисс между скоростью и размером файла, чтобы учесть и степень сжатия. Также на нем были показаны различные уровни сжатия для алгоритмов.

Результаты.

На рис. 7 показана скорость чтения, на рис. 8 — скорость записи. Каждый рисунок содержит две панели: левая — зависимость скорости от процента попаданий в кэш, правая — соотношение между пропускной способностью и размером файла при 97,5% попаданий.

При нулевой локальности `Compio` ожидаемо медленнее `Stdio` из-за накладных расходов на декомпрессию/компрессию и навигацию по B-дереву. С ростом попаданий скорость быстро увеличивается. Для чтения (рис. 7, слева) при >90% попаданий многие компрессоры (особенно LZ4 и Zstd) превосходят `Stdio`: данные уже распакованы и находятся в оперативной памяти, операция сводится к копированию в буфер пользователя. `Stdio` также буферизует данные, но вынужден выполнять системные вызовы, тогда как `Compio` в случае попадания в кэш полностью избегает обращения к диску и декомпрессии. Для записи (рис. 8, слева) порог превосходства выше — >99%; при 90–99% скорость сравнима со `Stdio`, но остаётся в 2–5 раз ниже, поскольку периодически

требуется сжатие и синхронизация изменённых блоков с диском.

Правые панели на обоих рисунках подтверждают, что наивысшую скорость даёт LZ4, однако ценой большого объёма файла. Zstd с уровнем сжатия 1 обеспечивает наилучший баланс: скорость близка к LZ4, а размер значительно меньше. Zlib и Brotli при высоких уровнях дают сильное сжатие, но заметно медленнее.

Вывод.

Кэш блоков превращает Compio из решения, уступающего Stdio на случайных запросах, в систему, способную при высокой локальности превзойти несжатый ввод-вывод (на чтение) и вплотную приблизиться к нему на запись, одновременно экономя дисковое пространство. Выбор компрессора позволяет гибко регулировать компромисс между скоростью и размером.

3.5. Влияние размера блока и алгоритма сжатия.

Мотивация.

Размер блока `block_size` напрямую влияет как на скорость операций, так и на итоговый размер архива. Цель эксперимента — исследовать совместное влияние размера блока и выбора алгоритма сжатия на производительность произвольной записи и степень сжатия, а также определить конфигурацию, обеспечивающую наилучший баланс.

Методика.

На тестовом HTML-файле (32 МиБ) выполнялось 1000 случайных операций записи; размер записываемого фрагмента для каждой операции выбирался из гамма-распределения с `shape = 4096`, `scale = 2` (средний размер 8192 байт). Позиции начала записи равномерно случайны. Эксперимент повторялся для разных значений `block_size` (от 1 КиБ до 256 КиБ) и для нескольких компрессоров: LZ4, Zstd, Zlib и Brotli. Для каждого алгоритма варьировался уровень сжатия (соответствующие точки на графике показаны разными оттенками одного цвета). Параметры `block_size_minimum` и

`block_size_maximum` всегда оставались жёстко привязанными к `block_size` как $\text{block_size}/4$ и $\text{block_size} \times 4$ соответственно. Чтобы изолировать эффект от кэширования блоков, `cache_size_blocks` выставлялся равным 1.

Результаты.

На рис. 9 каждая линия соединяет точки одного компрессора при разных размерах блока; направление вдоль линии соответствует уменьшению `block_size` (от больших блоков слева к малым справа).

Общая для всех алгоритмов тенденция такова: при переходе к очень большим блокам (левая часть графика) скорость записи значительно падает, тогда как размер файла уменьшается уже несущественно. При чрезмерно малых блоках (правая часть) размер архива резко возрастает, а скорость, достигнув максимума в районе 16–32 КиБ, начинает снижаться. Падение скорости при малых размерах блока объясняется ростом общего количества блоков, что ведёт к увеличению высоты В-дерева, числа аллокаций и вызовов компрессора.

Среди рассмотренных компрессоров наиболее сбалансированным оказывается Zstd с уровнем сжатия 1: в диапазоне `block_size` 4–16 КиБ его точки лежат вблизи Pareto-фронта, уступая по скорости только LZ4, но давая заметно меньший размер файла. LZ4 демонстрирует наивысшую скорость записи, однако ценой большего объёма архива. Brotli обеспечивает наилучшее сжатие, но заметно медленнее; Zlib проигрывает Zstd и по скорости, и по степени сжатия. Различия между алгоритмами стабильны и хорошо воспроизводятся, однако формальное статистическое сравнение не проводилось, так как дисперсия измерений мала.

Вывод.

Полученные данные показывают, что в рамках данного сценария (произвольная запись фрагментов со средним размером 8 КиБ) оптимальным является диапазон размеров блока 4–16 КиБ, а наилучший компромисс между скоростью и сжатием достигается при использовании Zstd с уровнем 1. Однако эти рекомендации не следует напрямую экстраполировать на другие

сценарии использования. В частности, разумно предположить, что оптимальный размер блока в общем случае будет близок к среднему размеру операций чтения/записи: при ином распределении или другом паттерне доступа (например, последовательная запись или крупные фрагменты) точка оптимума, скорее всего, сместится.

3.6. Анализ накладных расходов.

Структура размера архива.

Общий размер файла-контейнера можно представить как сумму трёх компонент:

1. *«Размер непрерывно сжатого файла»* — объём данных, который получился бы при сжатии всего файла одним блоком (непрерывным потоком);
2. *«Издержки блочного сжатия»* — избыточность, вызванная независимым сжатием каждого блока (потеря общего контекста);
3. *«Издержки метаданных библиотеки»* — В-дерево, заголовок, таблица файлов, служебные поля блоков.

Для того чтобы оценить вклад каждой части, из файла enwik9 (1 ГБ) выбиралось 29 непересекающихся регионов по 32 МиБ и для каждого были выполнены следующие замеры:

- данные сжимались алгоритмом сжатия Zstd (с уровнем сжатия 1) и вычислялся получившийся размер («размер непрерывно сжатого файла»);
- для различных размеров блока (от 256 байт до 128 КиБ) выполнялось следующее: те же самые данные бились на блоки соответствующего размера, каждый блок сжимался независимо, получившиеся размеры суммировались и из получившегося значения вычитался размер с прошлого шага, чтобы отдельно получить «издержки блочного сжатия»;

Размер блока (байт)	Накладные расходы (% от оригинала)	
	Блочное сжатие	Метаданные Compro
256	36,07 $\pm 7,20$	47,23 $\pm 0,00$
512	27,65 $\pm 7,16$	23,86 $\pm 0,00$
1024	20,35 $\pm 7,01$	12,18 $\pm 0,00$
2048	15,40 $\pm 7,11$	6,34 $\pm 0,00$
4096	11,02 $\pm 5,89$	3,41 $\pm 0,00$
8192	6,90 $\pm 3,30$	1,95 $\pm 0,00$
16384	3,91 $\pm 1,84$	1,21 $\pm 0,00$
32768	2,74 $\pm 0,87$	0,85 $\pm 0,00$
65536	1,49 $\pm 0,50$	0,68 $\pm 0,00$
131072	0,82 $\pm 0,30$	0,59 $\pm 0,00$

Таблица 4: Средние значения накладных расходов (% от исходного размера) и стандартные отклонения по 29 регионам enwik9 для различных размеров блока. Стандартное отклонение для архива Compro равно нулю, поскольку при фиксированном размере файла издержки метаданных определяются только размером блока и не зависят от сжимаемости данных (проверено экспериментально).

- с помощью библиотеки Compro строились архивы с разными block_size, вычислялся их размер и из них вычитались размеры с прошлого шага, чтобы отдельно получить «издержки метаданных библиотеки».

«Издержки блочного сжатия» и «издержки метаданных библиотеки» для каждого региона считались в процентах относительно его размера (32 МиБ), после чего по 29 регионам вычислялись среднее значение и стандартное отклонение.

Результаты.

На рис. 10 видно, что при размере блока 8 КиБ средние «издержки блочного сжатия» составляют около 6,9 %, а метаданные добавляют около 2,0 %. При очень маленьких блоках (256 байт) размер файла-контейнера превышает размер исходных данных, что делает такой архив неприемлемым. При увеличении блока как потери от блочного сжатия, так и доля метаданных убывают, стремясь к нулю.

Вывод.

Собственные накладные расходы библиотеки (метаданные) невелики и не являются узким местом. Основной «платой» за произвольный доступ выступает ухудшение сжатия из-за независимой обработки блоков. Диапазон 4–16 КиБ удерживает эти издержки в разумных пределах (5–12%), одновременно обеспечивая высокую скорость доступа, что согласуется с предыдущими рекомендациями.

3.7. Итоги экспериментов.

Проведённые измерения подтверждают, что архитектурные решения, заложенные в Compio, работают на практике:

- Механизм границ `block_size_min/max` успешно стабилизирует количество блоков при длительных сериях вставок и удалений, предотвращая падение производительности, характерное для наивного подхода.
- По скорости произвольного чтения Compio значительно превосходит Zran и практически не уступает Seekable Zstd (разница 3–8% по скорости, 4–5% по размеру архива), при этом предоставляя уникальную возможность модификации — записи, вставки и удаления — без полной перепакровки архива.
- Система кэширования делает библиотеку особенно эффективной при высокой локальности обращений, позволяя на чтении превосходить даже Stdio (без сжатия), а на записи достигать сравнимых скоростей.
- Пользователь может гибко настраивать компромисс «скорость – степень сжатия», выбирая компрессор и размер блока. Для сценария со случайными записями размера около 8 КиБ рекомендованный диапазон размера блока — 4–16 КиБ с компрессором Zstd-1.
- Накладные расходы метаданных библиотеки остаются в пределах 8,9% от исходного размера данных (при размере блока 8 КиБ), из которых 6,9% составляет потеря эффективности сжатия из-за блочной организации, то есть фундаментальное архитектурное ограничение.

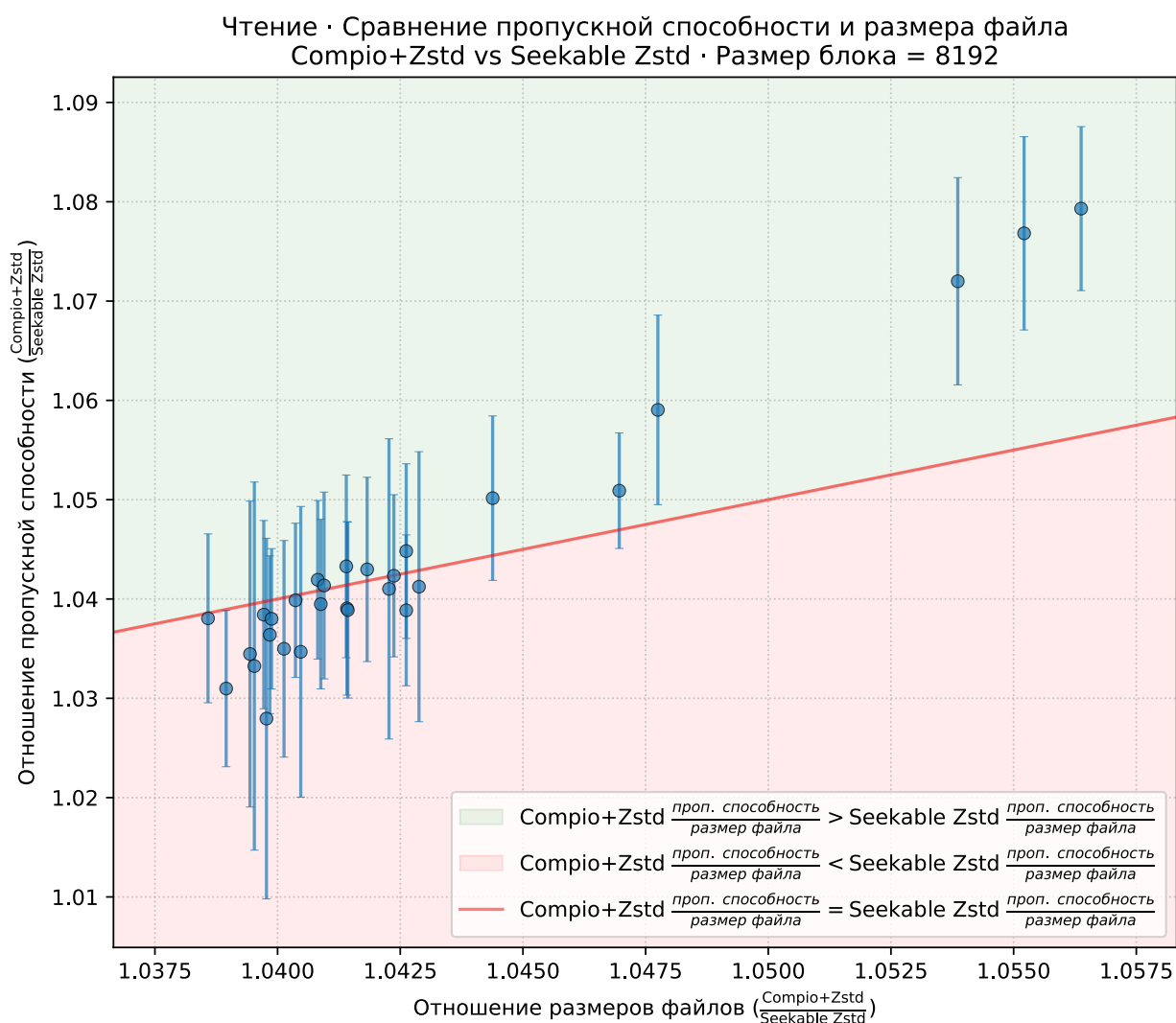


Рис. 6: Сравнение Compio+Zstd и Seekable Zstd на 29 регионах enwik9 при параметре granularity 8192 байт. Оси — отношения размеров архивов и скоростей чтения (Compio / Seekable Zstd). Красная линия $y = x$; зелёная область — у Compio выше отношение $\frac{\text{проп. способность}}{\text{размер файла}}$, розовая — у Seekable Zstd. Разница минимальна (3–8% по скорости, 4–5% по размеру архива).

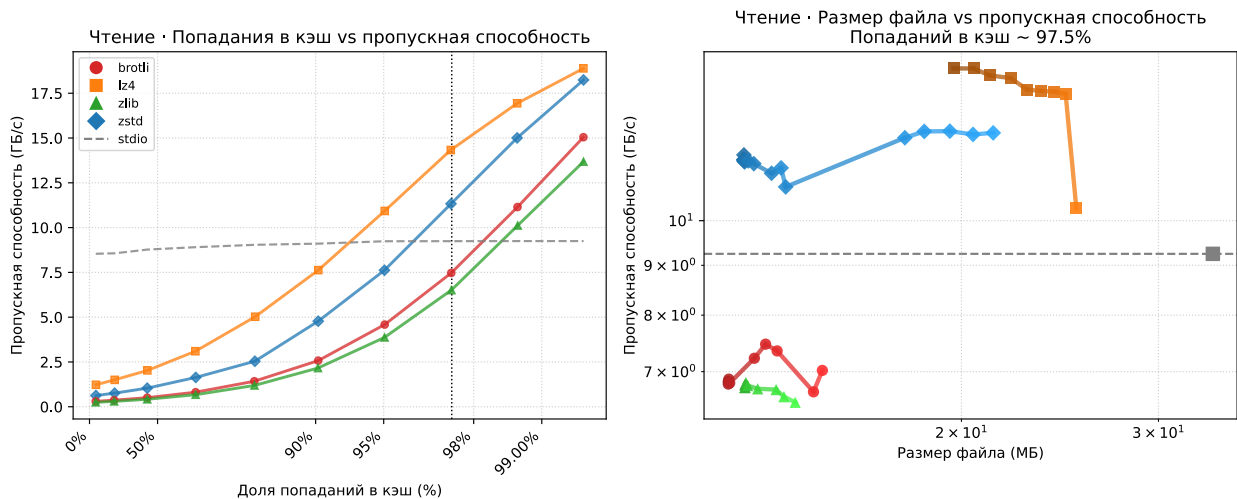


Рис. 7: Влияние кэша блоков на чтение. Левая панель: скорость чтения (ГБ/с) в зависимости от доли попаданий в кэш для Compio (алгоритмы LZ4, Zstd, Zlib, Brotli) и Stdio (baseline, ≈ 9 ГБ/с). При 90% попаданий Compio+LZ4 достигает 7,5 ГБ/с (немного уступает Stdio), Compio+Zstd — 5 ГБ/с (примерно вдвое ниже). При высокой локальности (>98%) LZ4 выходит на 18 ГБ/с, обгоняя Stdio. Правая панель: скорость чтения при фиксированной локальности 97,5% в зависимости от размера файла (логарифмическая шкала); более темный цвет соответствует более высокому уровню сжатия. LZ4 даёт 13–15 ГБ/с, но ценой большого размера архива. Zstd (11–12 ГБ/с) заметно выигрывает по размеру файла, обеспечивая лучший баланс (слева-сверху на графике).

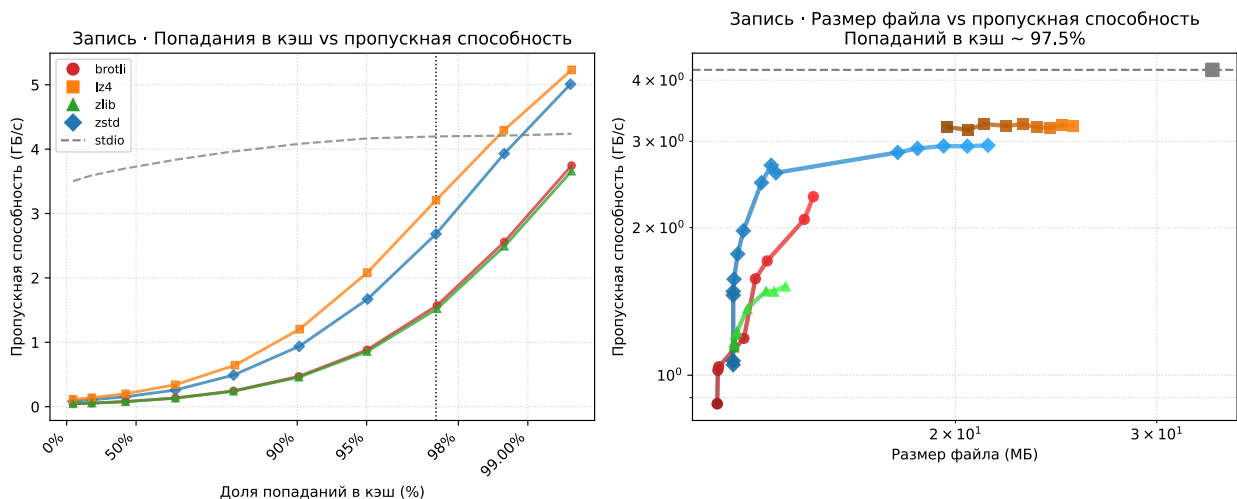


Рис. 8: Влияние кэша блоков на запись. Левая панель: скорость записи (ГБ/с) в зависимости от доли попаданий в кэш для Compio (алгоритмы LZ4, Zstd, Zlib, Brotli) и Stdio (baseline, скорость ≈ 4 ГБ/с). При 90% попаданий Compio+LZ4 — 1,2 ГБ/с, Compio+Zstd — 1 ГБ/с, что медленнее Stdio. Порог превосходства над Stdio достигается при >99% попаданий. Правая панель: скорость записи при фиксированной локальности 97,5% в зависимости от размера файла; более темный цвет соответствует более высокому уровню сжатия. LZ4 даёт 3,2 ГБ/с, Zstd — 2,7 ГБ/с. Как и в случае чтения, Zstd обеспечивает лучший баланс между скоростью и размером архива (слева-сверху на графике).

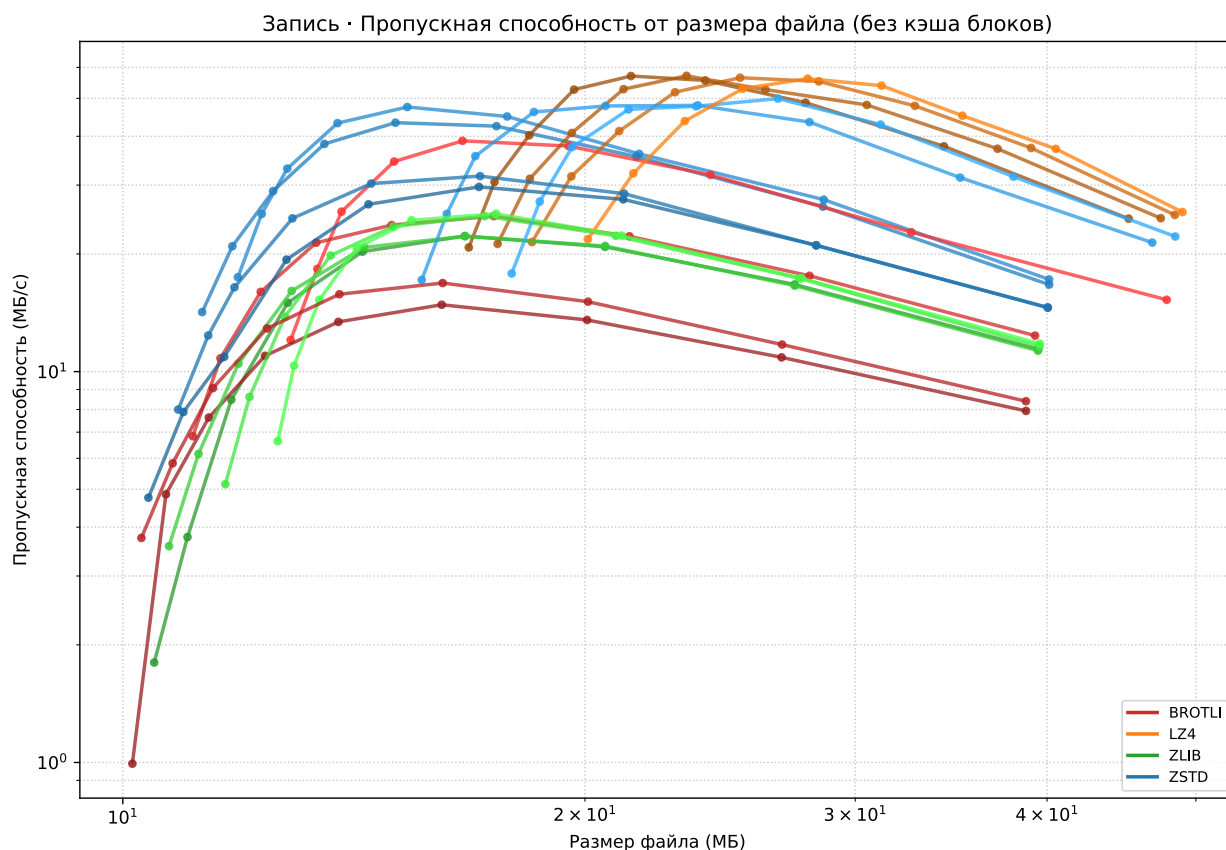


Рис. 9: Сравнение Compro при различных алгоритмах сжатия (LZ4, Zstd, Zlib, Brotli) и уровнях сжатия (более темный цвет соответствует более высокому уровню сжатия) для разных размеров блока (1–256 КиБ). Кэш блоков отключён. По осям — скорость произвольной записи (МБ/с) и размер архива (МБ). При малых блоках (<4 КиБ) размер архива резко растёт; при больших (>64 КиБ) скорость падает. В условиях данного эксперимента оптимальный диапазон — 4–16 КиБ. Zstd на уровне сжатия 1 обеспечивает лучший баланс (слева-сверху на графике): скорость, близкую к LZ4, и размер, близкий к Brotli.

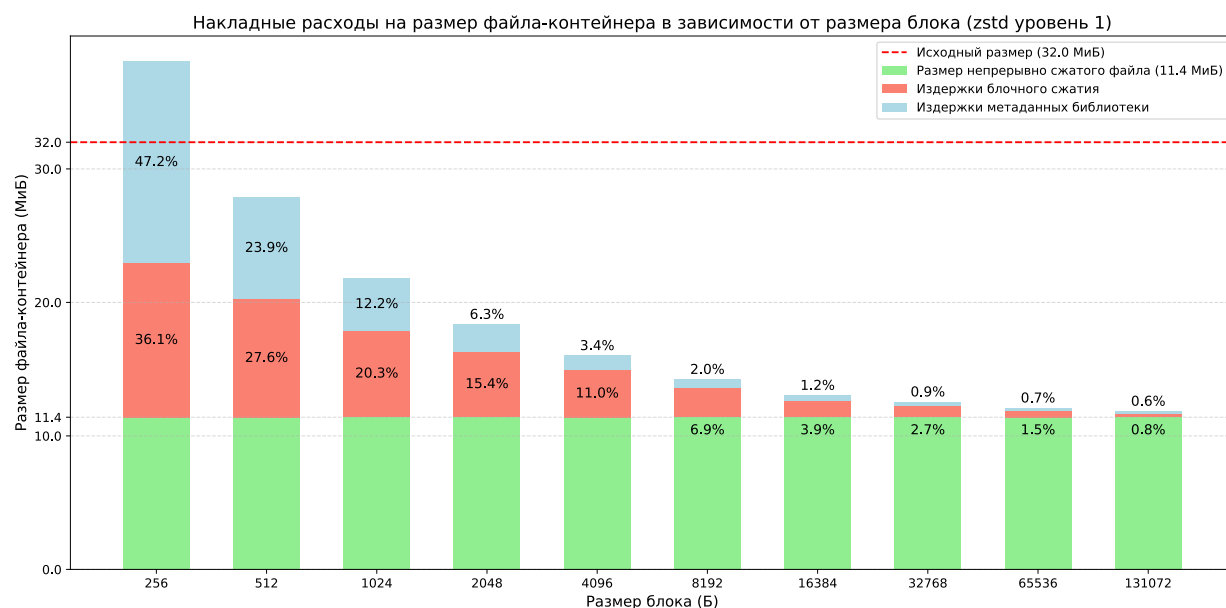


Рис. 10: Накладные расходы при блочном сжатии (Zstd, уровень 1) на 29 регионах enwik9 (по 32 МиБ) в зависимости от размера блока. Размер файла-контейнера складывается из трех величин: «размер непрерывно сжатого файла», «издержки блочного сжатия», «издержки метаданных библиотеки». 100% соответствует исходному размеру региона (32 МиБ). При размере блока 8 КиБ средние суммарные издержки составляют 8,9%.

Выводы.

В ходе выполнения выпускной квалификационной работы были получены следующие основные результаты.

1. Разработана библиотека `Compio`, предоставляющая интерфейс ввода-вывода, аналогичный стандартной библиотеке `Stdio` (функции `open`, `close`, `read`, `write`, `seek`, `tell`, а также `insert` и `erase`). Пользователь может использовать как поддерживаемые компрессоры (`Zlib`, `LZ4`, `Zstd`, `Brotli`), так и собственные реализации. Тип компрессора сохраняется в заголовке контейнера, что обеспечивает корректное открытие архива на любой системе.
2. Опубликована документация для внешнего API на C (см. 3.7), а также проведены тесты корректности ключевых компонентов (B-дерева, файловых операций, вставки/удаления) с использованием `Google Test` (подробнее в разделе 2).
3. Предложен и реализован самодостаточный формат файла-контейнера, поддерживающий многофайловость за счёт использования составного ключа в B-дереве (для индексирования всех логических файлов одним B-деревом) и за счёт аллокатора⁴, позволяющего упаковывать структуры произвольных размеров внутри одного файла.
4. Реализовано блочное сжатие с динамическим управлением размерами блоков. Механизм границ `block_size_minimum` и `block_size_maximum` эффективно предотвращает деградацию производительности из-за внутренней фрагментации при длительных последовательностях операций вставки и удаления. Экспериментально показано, что предложенная стратегия сохраняет количество блоков стабильным и поддерживает скорость операций вставки и удаления на уровне 20–30 МБ/с даже после деградации, связанной с внешней фрагментацией, в отличие от наивного подхода, где количество блоков неограниченно растёт, а скорость падает до 2–3 МБ/с.

⁴Аллокатор блоков разработан Д. В. Корбелайнен и детали его реализации выходят за рамки настоящей работы.

5. Для индексации блоков применено В-дерево, оптимизированное для работы с дисковой памятью. Помимо стандартных точечных операций, реализованы массовые операции: эффективное получение всех блоков в непрерывном диапазоне и массовый сдвиг ключей на заданное смещение. Последний выполняется через механизм отложенных операций с ленивым распространением смещений, работающий за амортизированное логарифмическое время. Такой подход не является стандартным для В-деревьев и критически важен для поддержки эффективных вставки и удаления.
6. Разработана система кэширования из двух независимых кэшей: кэш узлов В-дерева и кэш распакованных блоков данных. Кэш блоков хранит данные в распакованном виде, что позволяет при повторных обращениях полностью избегать операций сжатия и распаковки, существенно снижая вычислительную нагрузку.
7. Для согласованности кэша блоков с индексом при вытеснении изменённых блоков был придуман и реализован временный индекс — временная структура в оперативной памяти, хранящая актуальные адреса вытесненных блоков на время выполнения массовой операции и очищаемая после её завершения, которая позволяет избежать лишних запросов к В-дереву при операциях, затрагивающих множество блоков.
8. Исследовано влияние кэширования, размера блока и алгоритма сжатия на производительность *Compio*. В зависимости от локальности обращений скорость чтения варьируется от 626 МБ/с до 18,2 ГБ/с, скорость записи — от 86,5 МБ/с до 5,0 ГБ/с; при высокой локальности кэш распакованных блоков позволяет обогнать несжатый *Stdio* на чтении и вплотную приблизиться к нему на записи. Накладные расходы метаданных библиотеки не превышают 2% от исходного размера данных; основная плата за произвольный доступ — «издержки блочного сжатия» (около 6.9% при блоке 8 КиБ).
9. Проведено сравнение *Compio* с аналогами в сценарии произвольного чтения. В отличие от *Zran*, который эффективен лишь при очень круп-

ных запросах и при среднем размере запроса 8 КиБ показывает низкую скорость (<20 МБ/с), Compio позволяет настраивать размер блока под ожидаемый паттерн доступа. При оптимальном размере блока 8–16 КиБ (в условиях проведенного эксперимента) Compio даёт баланс, близкий к Seekable Zstd: Compio быстрее на 3–8%, но архив Seekable Zstd на 4–5% меньше. Таким образом, Compio не уступает специализированному инструменту для чтения, предоставляя при этом поддержку записи и модификации данных.

Заключение.

В рамках работы создана библиотека `Comrio`, решающая актуальную задачу эффективного хранения и модификации сжатых данных с произвольным доступом. В отличие от существующих аналогов, библиотека поддерживает не только чтение, но и полноценную запись, включая вставку и удаление фрагментов в середине логического файла, без необходимости полной перепаковки данных.

Ключевыми особенностями разработанного решения являются:

- универсальность — возможность подключения произвольных алгоритмов сжатия;
- самодостаточность формата файла-контейнера, не требующего внешних индексов;
- эффективное управление внутренней фрагментацией блоков, подтверждённое экспериментально;
- двухуровневое кэширование, минимизирующее операции ввода-вывода и сжатия;
- совместимость с любыми языками программирования через стабильный C ABI.

Практическая значимость библиотеки заключается в её применимости в областях, где необходимо хранить большие объёмы данных в сжатом виде и при этом регулярно выполнять операции произвольной записи или редактирования, — биоинформатика, астрономия, системы логирования и аналитики.

Дальнейшие направления развития включают разработку обёрток для языков высокого уровня (в первую очередь Python), экспериментальную проверку работы библиотеки на файлах существенно большего объёма (десятки и сотни гигабайт) для оценки масштабируемости алгоритмов, реализацию многопоточной обработки, использующей независимость сжатия отдельных блоков для дополнительного ускорения операций ввода-вывода, а также исследование возможности поиска непосредственно в сжатых данных.

Список литературы.

1. 7z Format. — 7-Zip. — URL: <https://www.7-zip.org/7z.html> ; Дата обращения: 23.05.2026.
2. *Adler M., zlib contributors.* zran.c: Example of deflate stream indexing and random access. — URL: <https://github.com/madler/zlib/blob/master/examples/zran.c> ; Дата обращения: 23.05.2026.
3. *Alakuijala J., Szabadka Z.* Brotli Compressed Data Format : тех. отч. / Internet Engineering Task Force (IETF). — 07.2016. — RFC 7932. — 10.17487/RFC7932.
4. *Apache Software Foundation.* Apache Parquet Format Specification. — 2013. — URL: <https://github.com/apache/parquet-format> ; Дата обращения: 23.05.2026.
5. *Bayer R., McCreight E. M.* Organization and maintenance of large ordered indexes // Acta Informatica. — 1972. — Т. 1, № 3. — С. 173—189. — 10.1007/BF00288683.
6. *Collet Y., Kucherawy M.* Zstandard Compression and the 'application/zstd' Media Type : тех. отч. / Internet Engineering Task Force (IETF). — 02.2021. — RFC 8878. — 10.17487/RFC8878.
7. *Collet Y.* LZ4: Extremely Fast Compression algorithm. — URL: <https://github.com/lz4/lz4> ; Дата обращения: 23.05.2026.
8. *Collet Y., Facebook.* Zstandard – Real-time data compression algorithm. — 2016. — URL: <https://github.com/facebook/zstd> ; Дата обращения: 23.05.2026.
9. *Deorowicz S.* Silesia Compression Corpus. — 2003. — URL: <http://sun.aei.polsl.pl/~sdeor/index.php?page=silesia> ; Дата обращения: 23.05.2026.
10. *Deutsch L. P.* DEFLATE Compressed Data Format Specification version 1.3 : тех. отч. / Internet Engineering Task Force (IETF). — 05.1996. — RFC 1951. — 10.17487/RFC1951.

11. Efficient storage of high throughput DNA sequencing data using reference-based compression / M. H.-Y. Fritz [и др.] // *Genome Research*. — 2011. — Т. 21, № 5. — С. 734—740. — 10.1101/gr.114819.110.
12. *Facebook*. Seekable Format for Zstandard. — URL: https://github.com/facebook/zstd/tree/dev/contrib/seekable_format ; Дата обращения: 23.05.2026.
13. *Fu J., Ke B., Dong S.* LCQS: an efficient lossless compression tool of quality scores with random access functionality // *BMC Bioinformatics*. — 2020. — Т. 21, № 1. — С. 109. — 10.1186/s12859-020-3428-7.
14. *Google*. Brotli: A generic-purpose lossless compression algorithm. — URL: <https://github.com/google/brotli> ; Дата обращения: 23.05.2026.
15. *Google*. Google Benchmark: A Microbenchmark Support Library. — URL: <https://github.com/google/benchmark> ; Дата обращения: 23.05.2026.
16. *Google*. GoogleTest: Google Testing and Mocking Framework. — URL: <https://github.com/google/googletest> ; Дата обращения: 23.05.2026.
17. *Johnson T., Shasha D.* 2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm // *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*. — San Francisco, CA, USA : Morgan Kaufmann, 1994. — С. 439—450. — URL: <http://www.vldb.org/conf/1994/P439.PDF> ; Дата обращения: 23.05.2026.
18. *Mahoney M.* Large Text Compression Benchmark. — URL: <https://mattmahoney.net/dc/text.html> ; Дата обращения: 23.05.2026.
19. RAR Archive Format. — RARLAB. — URL: <https://www.rarlab.com/> ; Дата обращения: 23.05.2026.
20. Tiled Image Convention for Storing Compressed Images in FITS Binary Tables / R. L. White [и др.]. — 2012. — arXiv: 1201.1336 [astro-ph.IM]. — URL: <https://arxiv.org/abs/1201.1336> ; Дата обращения: 23.05.2026.

21. XZ Utils / L. Collin [и др.]. — Вер. 5.6.0. — The Tukaani Project, 2024. — URL: <https://tukaani.org/xz/> ; Дата обращения: 23.05.2026.
22. zlib: A Massively Spiffy Yet Delicately Unobtrusive Compression Library. — URL: <https://zlib.net/> ; Дата обращения: 23.05.2026.

Приложение А. Ссылки на исходный код и документацию

Ссылка на GitHub репозиторий

Репозиторий проекта на GitHub доступен по адресу: <https://github.com/lovyagin/compio> (дата обращения: 23.05.2026).

Ссылка на опубликованную документацию

Документация для внешнего API для языка C, сгенерированная с помощью Doxygen, доступна по адресу: <https://lovyagin.github.io/compio/> (дата обращения: 23.05.2026).